EventForce Management System

Complete Project Report

Salesforce Project Report

Team ID: SWTID-2026-1405 | SmartBridge x SkillWallet

Team Leader: Monika P

Team Members: Usha S, Shobana M, Mujahidha Begam M

P.B. College of Engineering, B.Tech Information Technology

Contents

No	Section
1	Ideation Phase
2	Requirement Analysis
3	Project Design Phase
4	Project Planning and Scheduling
5	Development Phase 1: Requirement Analysis and Planning
6	Development Phase 2: Backend Development and Configurations
7	Development Phase 3: UI/UX Development and Customization
8	Development Phase 4: Data Migration, Testing and Security
9	Testing Results and Conclusion

This report was written by the team from its own analysis, design and implementation work. All screenshots are taken from the team's Salesforce Developer Edition org.

Ideation Phase

1. Project overview

EventForce is a Salesforce based event management system built for an event planning company. The company handles weddings, corporate meetings, birthdays, anniversaries, festivals and concerts. For each event it must coordinate a client, a venue and several vendors such as caterers, decorators and photographers. Our team decided to bring all of this into one Salesforce application so that every event, client, venue, vendor and feedback record is stored, linked and reported in a single place.

2. Problem statement

Small and medium event companies usually run their work through spreadsheets, phone calls and messaging apps. During our analysis we identified the following problems:

- The same venue can be promised to two clients on the same date because nobody has a single live view of bookings.
- Clients are not reminded before the event, which causes last minute confusion.
- Event cancellations are handled verbally, so there is no record of who approved the cancellation and when.
- The availability of a venue is not updated when an event is confirmed or cancelled.
- Customer feedback is scattered and cannot be linked to the event that it belongs to.
- Management has no quick summary of upcoming events and the budget planned for each month.

3. Stakeholders and their needs

Stakeholder	Need
Event Admin (owner)	Full control of the system, approves cancellation requests, sees dashboards
Event Coordinator	Creates and manages events, sees clients, venues and vendors, collects feedback
Vendor Manager	Maintains vendor records and sees the events that need vendors
Client	Sees the events booked for them and submits feedback after the event

4. Brainstorming and idea selection

The team listed possible ways to solve the problem and compared them on effort, cost, security and reporting.

Idea	Advantages	Drawbacks	Decision
Shared spreadsheet with email reminders	No setup cost	No validation, no access control, manual reminders, no approval trail	Rejected
Custom web application	Full flexibility	Needs hosting, login, security and reporting to be built from zero; long development time	Rejected
Salesforce Lightning application	Objects, security, approvals, automation, reports and dashboards are available on the platform; fast to build and easy to extend	Needs a Salesforce org and platform knowledge	Selected

5. Proposed solution

We will build EventForce on the Salesforce platform with six custom objects (Event, Client, Vendor, Venue, Feedback and the Event Vendor junction object). Business rules are enforced with a validation rule and Apex triggers. A record-triggered flow sends reminders, an approval process controls cancellations, and scheduled Apex keeps event statuses up to date. A Lightning app, a report and a dashboard give users and managers a clear view of the work.

6. Objectives

- Store events, clients, venues, vendors and feedback in linked custom objects.
- Prevent double booking of a venue on the same date.
- Keep venue availability correct when an event is confirmed or cancelled.
- Remind clients automatically three days before a confirmed event.
- Control event cancellation through a formal approval with email notifications.
- Close past events automatically every night.
- Give each user group only the access it needs.
- Provide a monthly view of upcoming events and planned budget.

7. Scope

In scope	Out of scope
Custom objects, fields, relationships, validation, approval, flow, Apex, security, Lightning app, report, dashboard, sample data and testing in a Developer Edition org	Payment handling, public booking website, mobile app, integration with external systems, production deployment

8. Why Salesforce

Salesforce gives a ready security model (profiles, roles, permission sets, sharing), declarative automation (validation rules, flows, approvals) and programmatic control (Apex). Reports and dashboards are built in. This lets a small student team deliver a working business application in a short time while learning the skills that are used in the industry.

Requirement Analysis

1. Functional requirements

ID	Requirement	Priority
FR-01	Store events with type, date, status, budget, venue and client	High
FR-02	Store client details with a valid email address, phone, address, country and city	High
FR-03	Store venue details with address, map link, capacity and availability status	High
FR-04	Store vendor details with service types, contact details and status	High
FR-05	Link many vendors to many events through a junction object	Medium
FR-06	Capture feedback with rating and comments for an event and its client	Medium
FR-07	Reject a client record that has an invalid email address	High
FR-08	Block two events at the same venue on the same date	High
FR-09	Set a venue to Reserved when an event is Confirmed and to Available when it is Canceled	High
FR-10	Send the client a reminder email three days before a confirmed event	High
FR-11	Approve or reject event cancellation with automatic status update and email	High
FR-12	Mark past events as Completed every night	Medium
FR-13	Calculate a default event budget from the event type	Low
FR-14	Provide a report and dashboard of upcoming events by month	Medium
FR-15	Restrict access by profile, role, permission set and sharing rule	High

2. Non-functional requirements

Area	Requirement
Usability	All objects reachable from one Lightning app with tabs; simple record forms
Security	Least privilege access; Event records private by default and shared only through a rule
Data quality	Picklists, required fields, validation rule and unique venue and date check
Reliability	Automation runs on the Salesforce platform (flows, triggers, scheduled jobs)
Maintainability	Business logic separated into a helper class and small triggers; configuration documented
Performance	Bulk safe triggers (queries outside loops) and batch processing for nightly jobs

3. Data requirements

Object	Purpose	Key fields
Event	Main record for each planned event	Event Name, Event Type, Event Date, Event Status, Event Budget, Venue, Client
Client	Customer who books events	Client Name, Email, Phone, Address, Country, City
Venue	Place where an event is held	Venue Name, Address, Location, Capacity, Availability Status
Vendor	Service provider for events	Vendor Name, Service Type, Email, Phone, Status
Event Vendor	Junction between events and vendors	Event (master-detail), Vendor (master-detail)
Feedback	Client opinion after an event	Rating, Comments, Event, Client

4. User roles and access matrix

C = Create, R = Read, E = Edit, D = Delete. The Feedback Manager permission set adds full access to Feedback for the Event Coordinator.

Profile	Event	Client	Vendor	Venue	Feedback
Event Admin	CRED	CRED	CRED	CRED	CRED
Event Coordinator	CRED	RE	R	R	CR (+ permission set)
Vendor Manager	R	R	CRED	R	R
Client Profile	R	-	No access	No access	CR

5. Technology stack

Layer	Technology
Platform	Salesforce Developer Edition
Data model	Custom objects, fields, relationships, formula fields, picklists
Business logic	Validation rule, Approval Process, Record-triggered Flow, Apex triggers, Batch and Schedulable Apex
User interface	Lightning App, tabs, reports and dashboards
Security	Profiles, roles, permission set, organization wide defaults, sharing rule
Data loading	Data Import Wizard and anonymous Apex
Tools	Setup, Object Manager, Flow Builder, Apex editor, GitHub

6. Assumptions and constraints

- The project is built in a free Developer Edition org, which has a limited number of full Salesforce licenses; the extra users use the Salesforce Platform license.
- The Data Import Wizard cannot map custom lookup fields, so Venue and Client on events are linked with a small Apex script after import.
- Email delivery depends on the org email settings; sample client addresses are fictitious and are used for demonstration only.

Project Design Phase

1. Solution architecture

EventForce follows the usual layered structure of a Salesforce application.

Layer	Components
Presentation	Event Planner Lightning app, object tabs, record pages, report and dashboard
Business logic	Validation rule on Client, Apex triggers on Event, record-triggered flow, approval process, scheduled batch job
Data	Event, Client, Venue, Vendor, Event Vendor and Feedback objects with relationships
Security	Profiles, role hierarchy, permission set, organization wide defaults and sharing rule

2. Data model

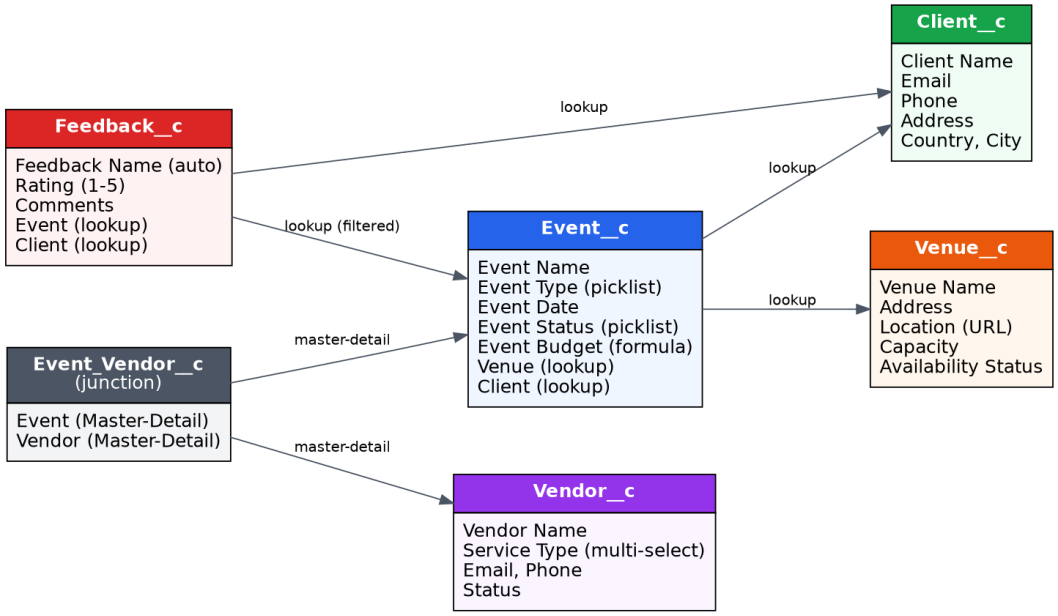


Figure 1: Object relationship diagram

Relationship	Type	Reason
Event to Venue	Lookup	A venue can host many events; an event may exist before a venue is final
Event to Client	Lookup	A client can book many events
Event Vendor to Event and Vendor	Master-Detail (two)	Many-to-many link: an event uses many vendors and a vendor serves many events
Feedback to Event	Lookup with filter	Only events of the selected client are offered
Feedback to Client	Lookup	Shows who gave the feedback

3. Process design

3.1 Automation overview

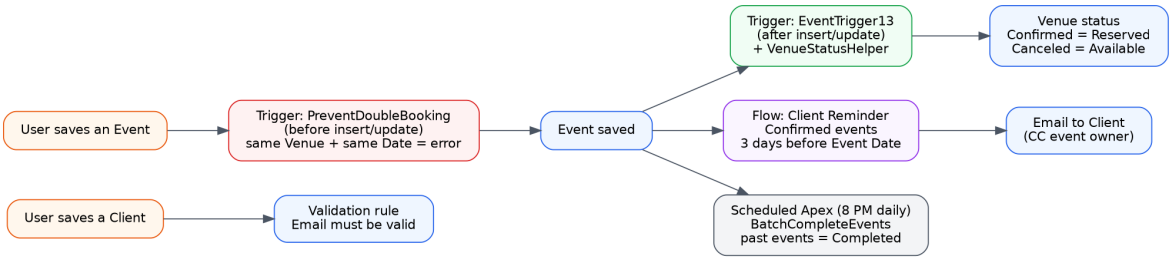


Figure 2: How validation, triggers, flow and scheduled Apex work together

3.2 Event cancellation approval

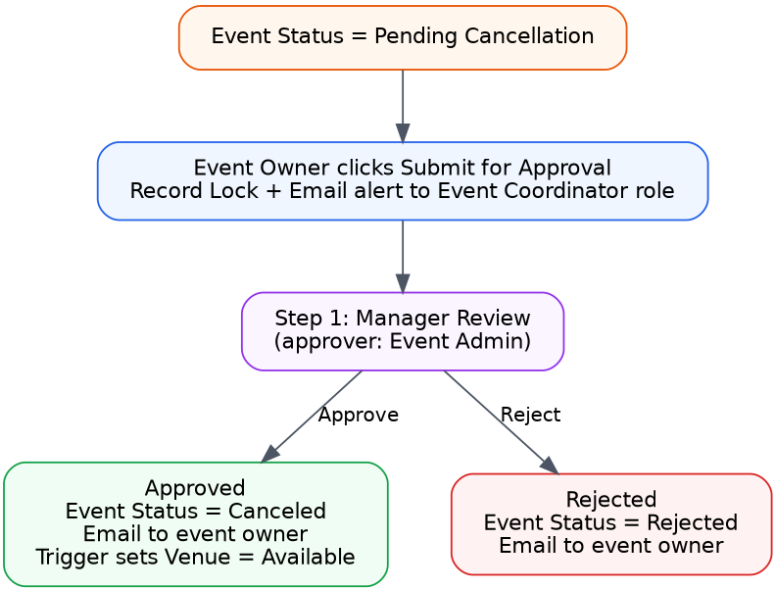


Figure 3: Cancellation approval process

4. User stories

As a	I want to	So that
Event Coordinator	create an event for a client at a venue	the booking is recorded and the budget is estimated
Event Coordinator	be stopped when I choose a venue that is already booked	no two clients get the same venue and date
Client	receive a reminder three days before my event	I am prepared for the day
Event Coordinator	request cancellation of an event	the admin can review it and the record is locked meanwhile
Event Admin	approve or reject the request	the event status and venue availability update automatically
Vendor Manager	maintain vendor details	the coordinators always see current vendor information
Event Admin	see upcoming events grouped by month	I can plan resources and budget

5. Security design

- Role hierarchy: Event Admin on top with Event Coordinator, Vendor Manager and Client below it.

- Four custom profiles give object level access; the Feedback Manager permission set gives extra Feedback access to the coordinator.
- Event is Private in the organization wide defaults. A sharing rule shares events owned by the Event Coordinator role with the Vendor Manager role as read only.
- Only the Event Admin approves cancellation requests.

6. Design decisions

- A junction object with two master-detail relationships is the standard way to model many-to-many in Salesforce.
- Venue update logic is placed in a helper class so the trigger stays small and the code can be reused.
- The double booking check runs before save, so invalid records never reach the database.
- Reminders use a scheduled path in a flow, which avoids custom code for a time based action.

Project Planning and Scheduling

1. Approach

The team followed a sprint based approach. Work was divided by module so that each member could build and test one part and then hand over to the next. The team leader coordinated the sprints, reviewed the build and maintained the documentation and the GitHub repository.

2. Sprint plan

Sprint	Goal	Owner	Deliverables	Duration
Sprint 1	Planning and design	Monika P	Ideation, requirement analysis, design and plan documents	Day 1 to 2
Sprint 2	Data model	Usha S	Developer org, objects, tabs, fields, relationships, formula, validation rule	Day 2 to 3
Sprint 3	Automation	Shobana M	Email templates, approval process, flow, profiles, roles, users	Day 3 to 5
Sprint 4	Apex and security	Mujahidha Begam M	Triggers, helper class, batch and scheduled Apex, permission set, OWD, sharing rule	Day 4 to 6
Sprint 5	UI and data	Monika P and Usha S	Lightning app, report, dashboard, sample data import	Day 6 to 7
Sprint 6	Testing and release	All members	Test cases, screenshots, report, demo video, GitHub repository	Day 7 to 8

3. Responsibility matrix

Member	Role	Modules
Monika P	Team Leader	Ideation, design and planning, Lightning app, report, dashboard, final testing, demo video, GitHub
Usha S	Team Member	Custom objects, tabs, fields, relationships, formula field, validation rule, sample data
Shobana M	Team Member	Email templates, approval process, flow, profiles, roles, users
Mujahidha Begam M	Team Member	Apex triggers and classes, batch and scheduled Apex, permission set, OWD, sharing rule, test cases

4. Milestones

No	Milestone	Status
1	Developer org and project setup	Completed
2	Objects, tabs, fields and relationships	Completed
3	Validation rule and formula field	Completed
4	Email templates and approval process	Completed
5	Flow for client reminder	Completed
6	Apex triggers, batch and scheduled job	Completed
7	Lightning app, report and dashboard	Completed
8	Profiles, roles, users, permission set and sharing	Completed

No	Milestone	Status
9	Data import and linking	Completed
10	Testing, documentation and GitHub	Completed

5. Risks and mitigation

Risk	Impact	Mitigation
Not enough Salesforce user licenses in the Developer org	Cannot create all users	Use the Salesforce Platform license for non admin users
Sharing recalculation blocks changing organization wide defaults	Delay in security setup	Wait for the background job to finish and retry
Lookup fields cannot be imported with the wizard	Events without venue or client	Link records with an anonymous Apex script
Org becomes inactive	Org lock	Log in regularly until the project is submitted

Development Phase 1: Requirement Analysis and Planning

This phase prepared the technical ground for the build. The requirements from the analysis phase were converted into a build order, naming standards and a working environment.

1. Environment setup

- Created a free Salesforce Developer Edition org and verified the account through email.
- Opened Setup, Object Manager and Flow Builder to confirm that all required tools are available.
- A GitHub repository holds the documents, code and screenshots of the project.

2. Naming standards

Item	Standard	Example
Custom objects	Singular business name	Event, Client, Venue, Vendor, Feedback, Event Vendor
Fields	Clear label, API name generated by Salesforce	Event Status (Event_Status__c)
Apex	Class names describe the job; triggers named by object	VenueStatusHelper, PreventDoubleBooking
Flows and processes	Name states the purpose	Client Reminder - 3 Days Before, Event_Cancellation_Process

3. Build order

Step	Work	Depends on
1	Objects and tabs	Org ready
2	Fields, formula and relationships	Objects
3	Validation rule	Client fields
4	Roles, profiles and users	Objects (for profile permissions)
5	Email templates and approval process	Event fields, users
6	Flow	Event and Client fields
7	Apex classes and triggers, batch job	Event and Venue fields
8	Lightning app, report, dashboard	Data and objects
9	Permission set, OWD and sharing rule	Roles, users and profiles
10	Data import, testing, documentation	All of the above

4. Readiness checklist

- Requirements reviewed and accepted by the team leader.
- Responsibilities assigned to the four members.
- Object and field list agreed before building so that API names stay stable.
- Test scenarios drafted in advance (validation, double booking, venue status, approval, scheduled job).

Development Phase 2: Backend Development and Configurations

This phase created the data model and all business logic of EventForce: objects, fields, relationships, validation, email templates, the approval process, the flow and the Apex code.

1. Custom objects and tabs

Six custom objects were created from Setup, Object Manager. Reports, activities, field history tracking (on Event) and search were enabled so that the objects work in the app, in reports and in global search. A custom tab was created for each object so users can open them from the Lightning app.

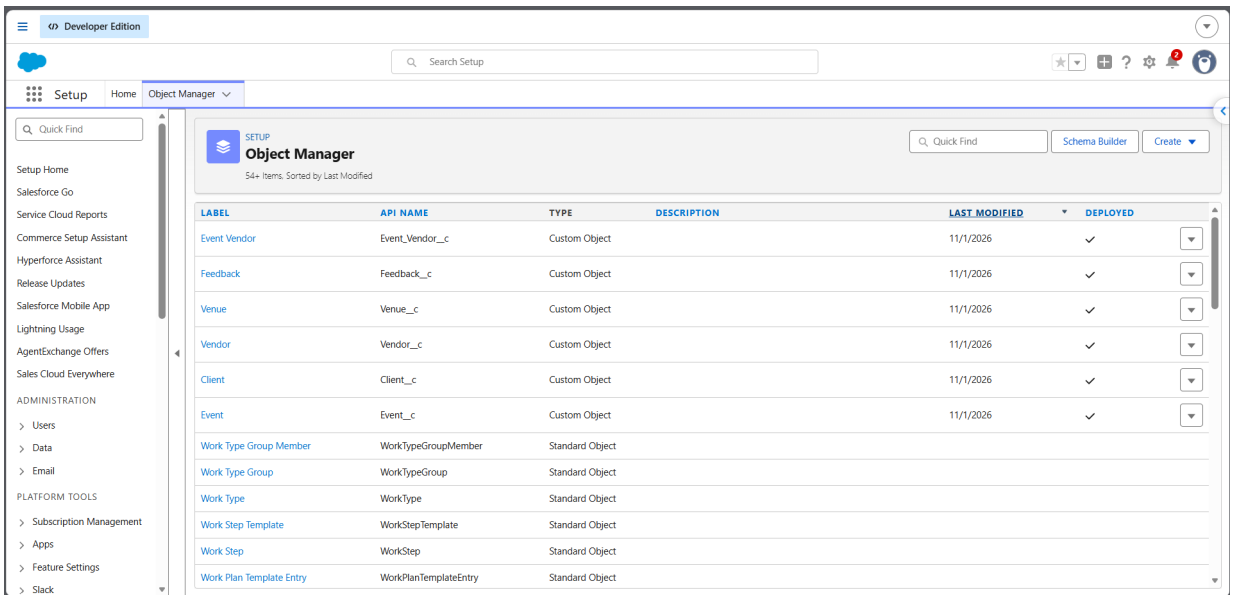


Figure 1: Custom objects in Object Manager

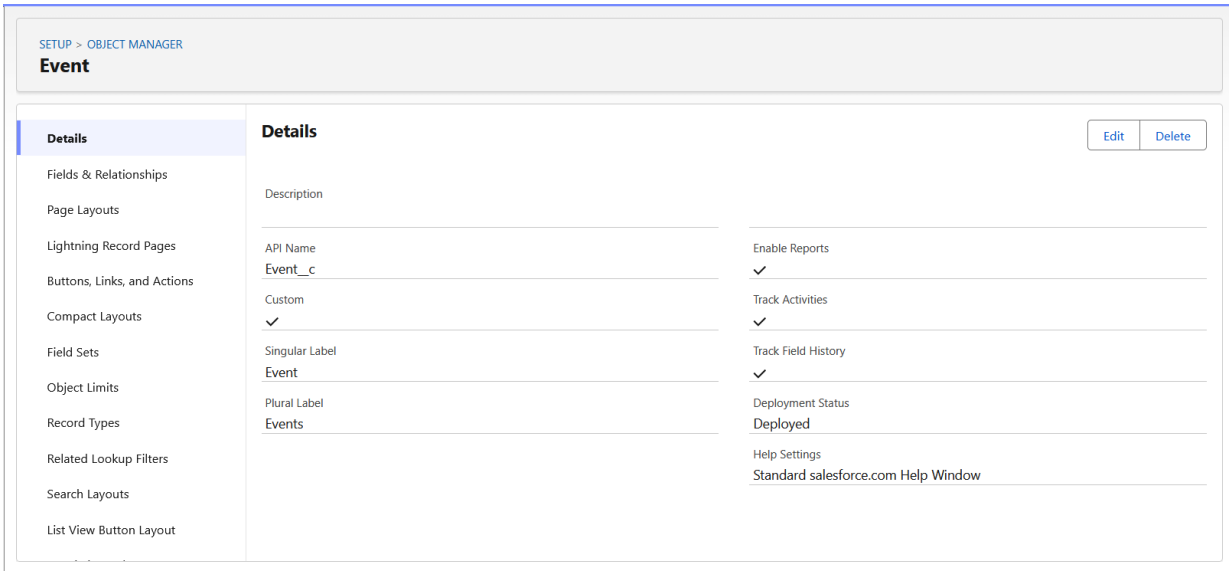


Figure 2: Event object definition (reports, activities and history tracking enabled)

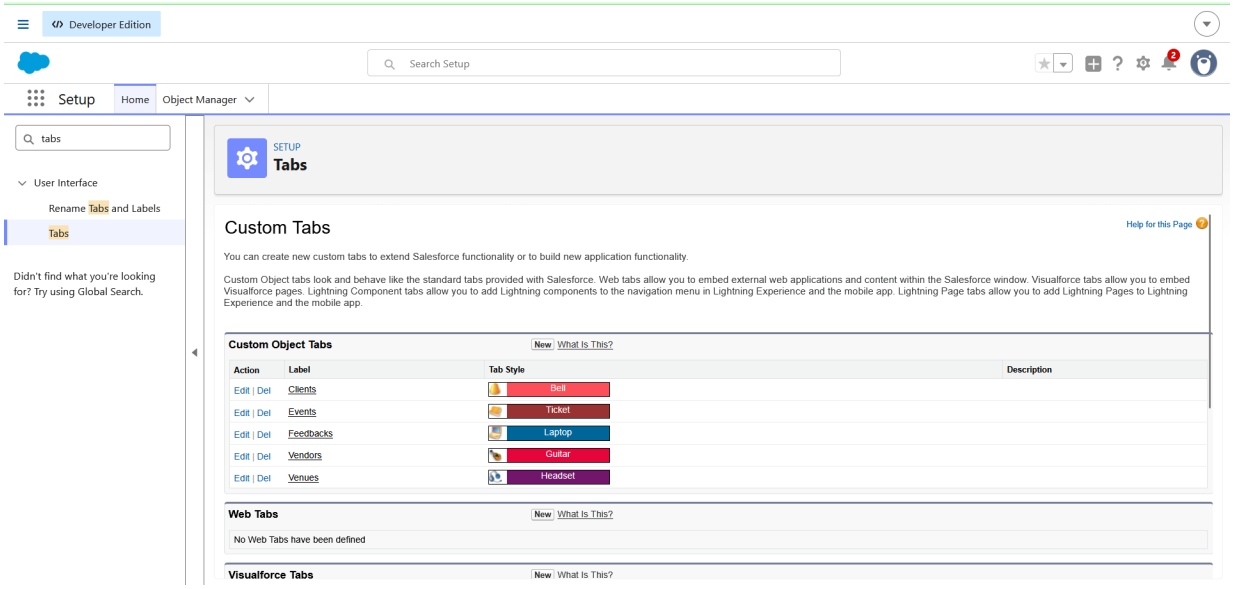


Figure 3: Custom tabs for Clients, Events, Feedbacks, Vendors and Venues

2. Fields

Event

Field	Type	Details
Event Name	Text (80)	Standard name field
Event Type	Picklist	Wedding, Corporate, Birthday, Anniversary, Festival, Concert, Other
Event Date	Date	Date of the event
Event Status	Picklist	Planned, Confirmed, Completed, Pending Cancellation, Canceled, Rejected
Event Budget	Formula (Currency)	Default budget from the event type
Venue	Lookup (Venue)	Where the event is held
Client	Lookup (Client)	Who booked the event

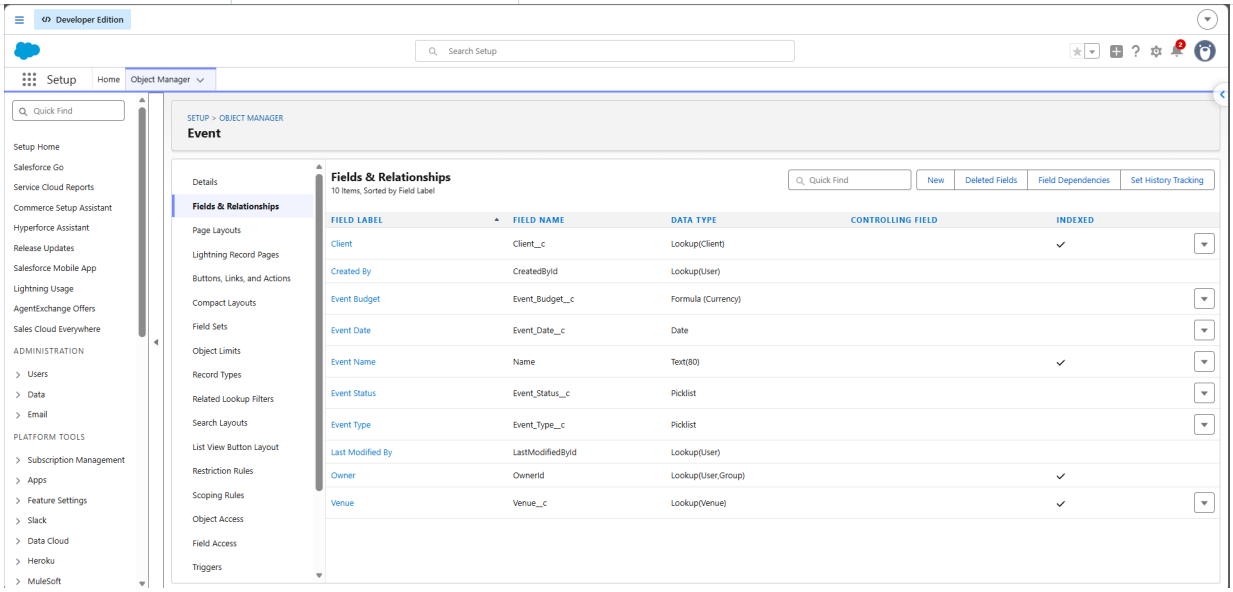


Figure 4: Event fields and relationships

Client, Venue, Vendor and Feedback

Object	Fields
Client	Email (Email), Phone (Phone), Address (Text Area), Country (Picklist), City (Picklist)
Venue	Address (Text Area), Location (URL), Capacity (Number), Availability Status (Available, Reserved, Not Available)
Vendor	Service Type (Multi-select: Catering, Decor, Photography, Videography, Lighting, Stage Setup, Makeup Artist, DJ/Music, Transportation, Hosting/Anchor), Email, Phone, Status (Available, Booked, Cancelled)
Feedback	Rating (1 to 5), Comments (Long Text Area), Event (Lookup), Client (Lookup). Record name is an auto number F-{0000}

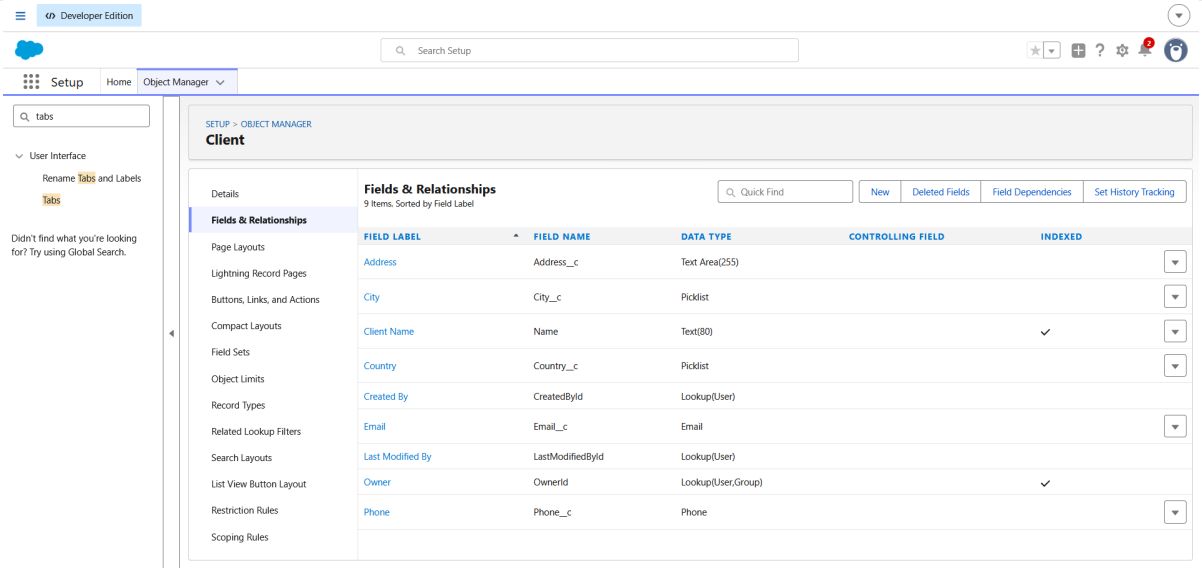


Figure 5: Client fields

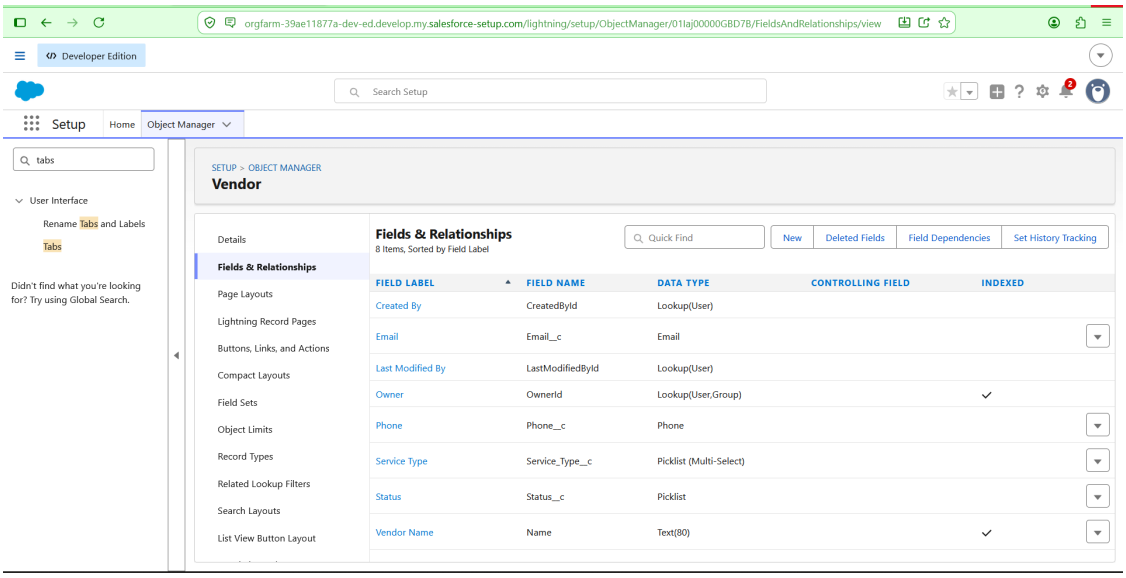


Figure 6: Vendor fields

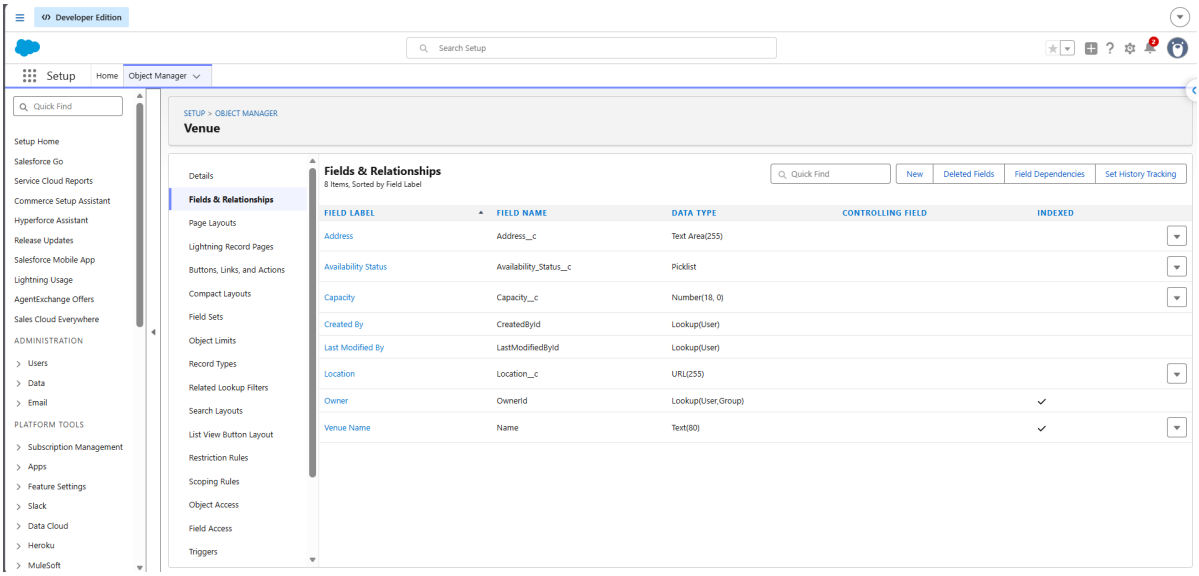


Figure 7: Venue fields

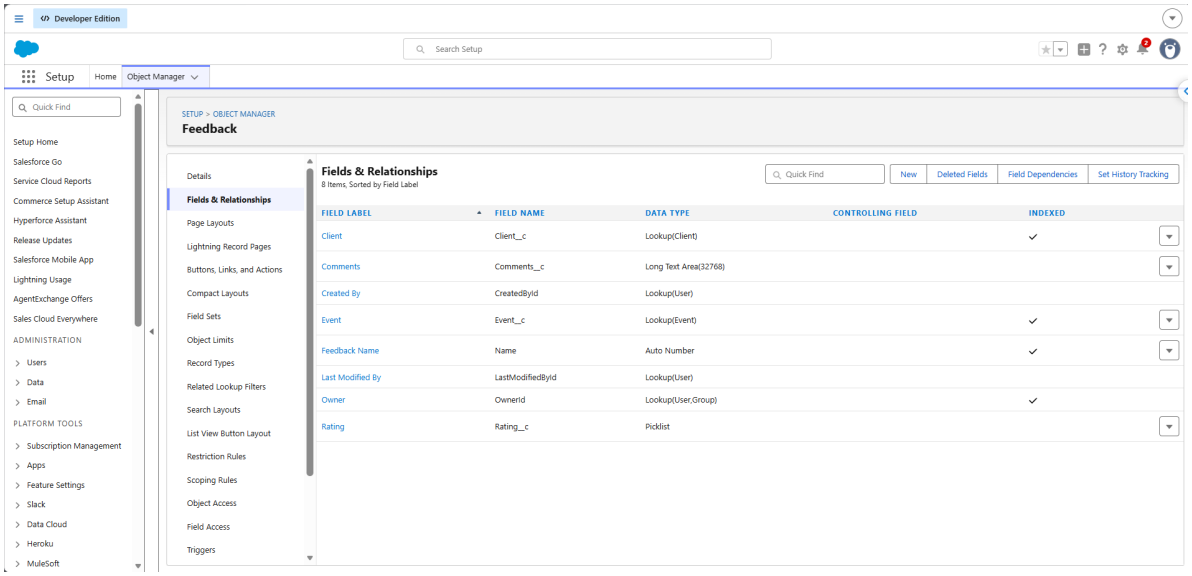


Figure 8: Feedback fields with Event and Client lookups

3. Relationships

Event Vendor is a junction object. It has two master-detail relationships, one to Event and one to Vendor, which creates the many-to-many link between events and vendors. On Feedback, a lookup filter on the Event field only offers events whose client is the same as the client chosen on the feedback record.

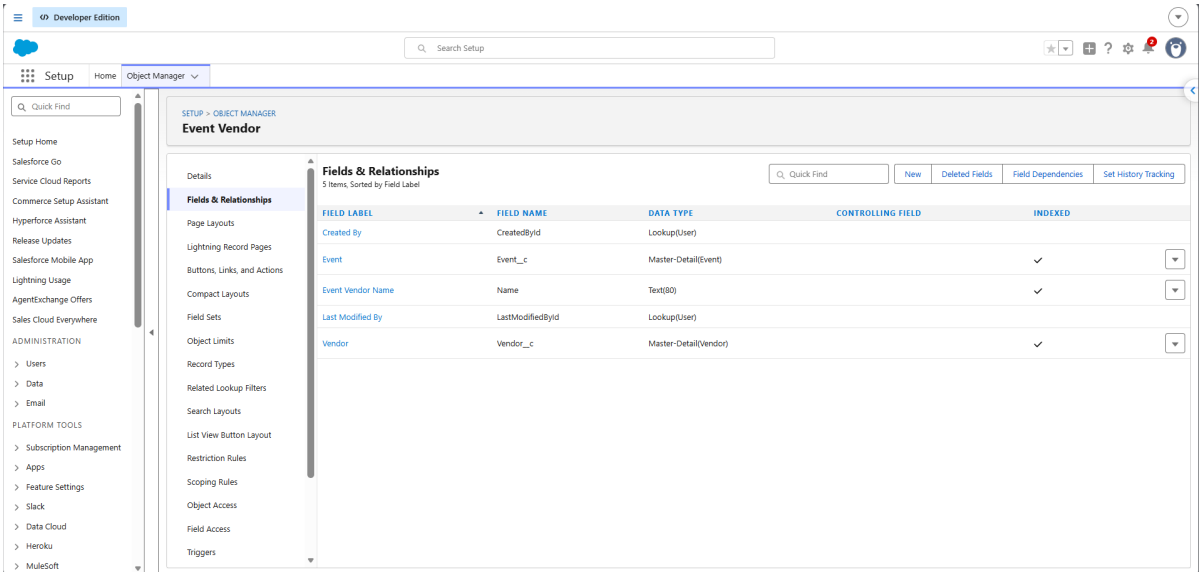


Figure 9: Event Vendor junction object with two master-detail fields

4. Formula field

Event Budget gives a default budget based on the event type.

```
CASE(TEXT(Event_Type__c),
  "Wedding", 50000, "Corporate", 30000, "Birthday", 10000, "Anniversary", 20000,
  "Festival", 60000, "Concert", 40000, "Other", 15000, 0)
```

5. Validation rule

The rule Email_Valid_Address on Client shows the message Please Enter Valid Email Address on the Email field when the address does not match the expected pattern.

```
NOT(REGEX(Email__c, "^[a-zA-Z0-9._]+@[a-zA-Z0-9.]+\\.[a-zA-Z]{2,}$"))
```

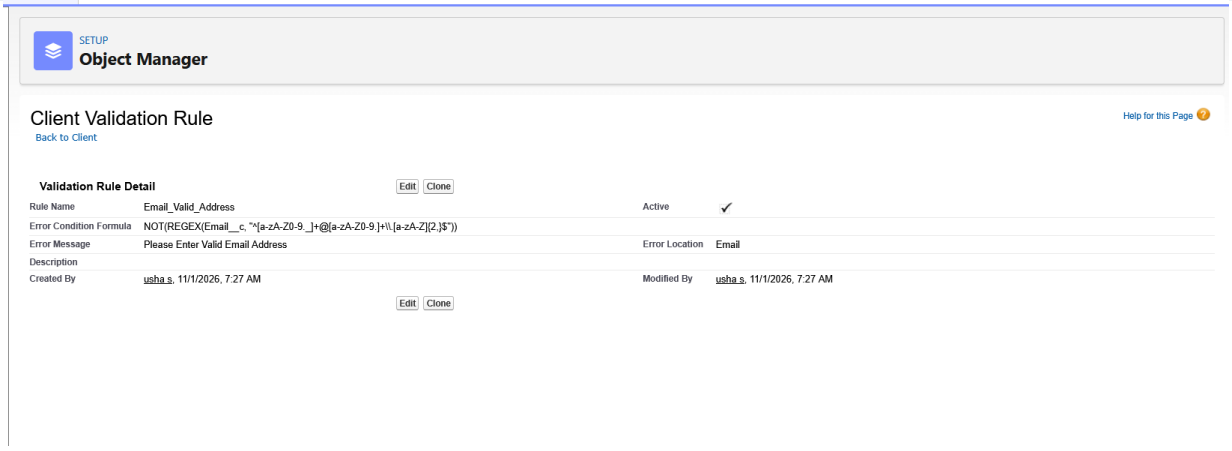


Figure 10: Validation rule on Client

6. Email templates

Four text email templates support the approval process: Event Cancellation Request Notification (sent to the Event Coordinator role on submission), Email to Client After Cancellation, Approval Notification to Event Owner and Rejection Notification to Event Owner. They use merge fields such as the event name, event date and owner name.

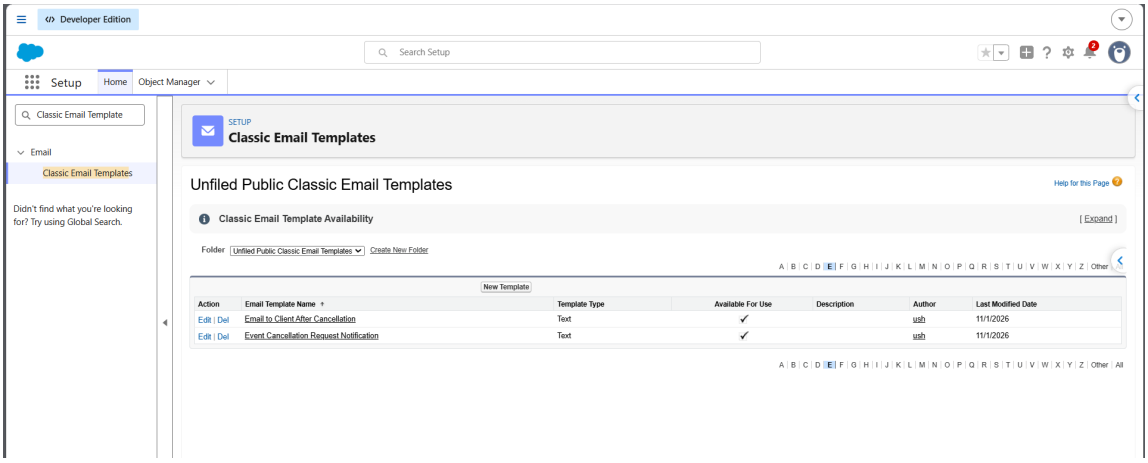


Figure 11: Email templates starting with E

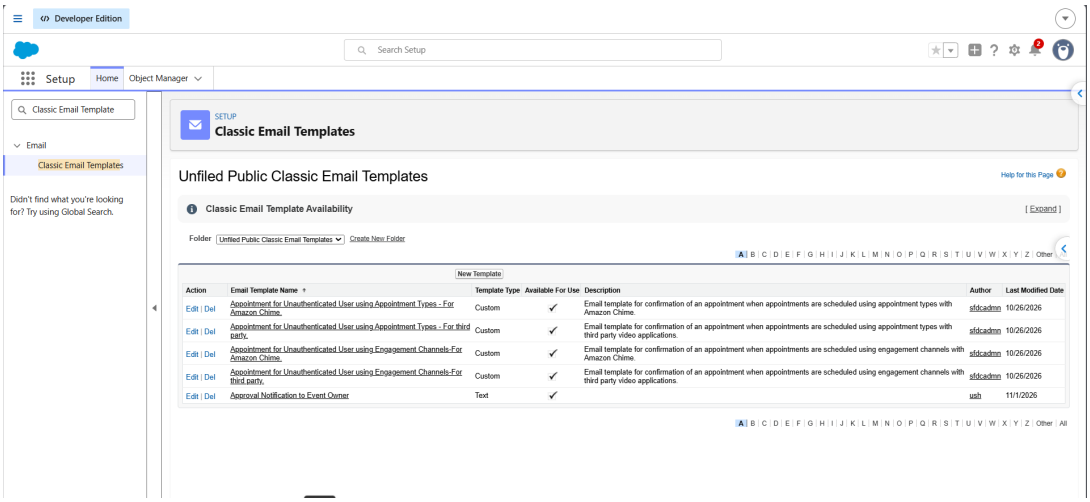


Figure 12: Approval Notification to Event Owner

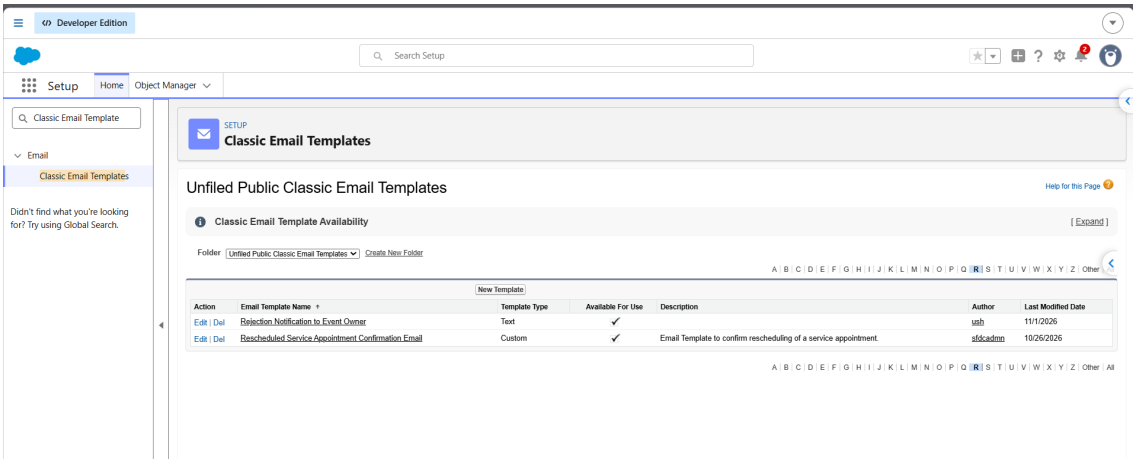


Figure 13: Rejection Notification to Event Owner

7. Approval Process

Part	Configuration
Name	Event_Cancellation_Process on Event
Entry criteria	Event Status equals Pending Cancellation
Initial submitters	Event Owner
Initial submission actions	Record lock and email alert to the Event Coordinator role

Part	Configuration
Approval step	Manager Review, approver is the Event Admin user
Final approval actions	Event Status set to Canceled and email alert to the event owner
Final rejection actions	Event Status set to Rejected and email alert to the event owner

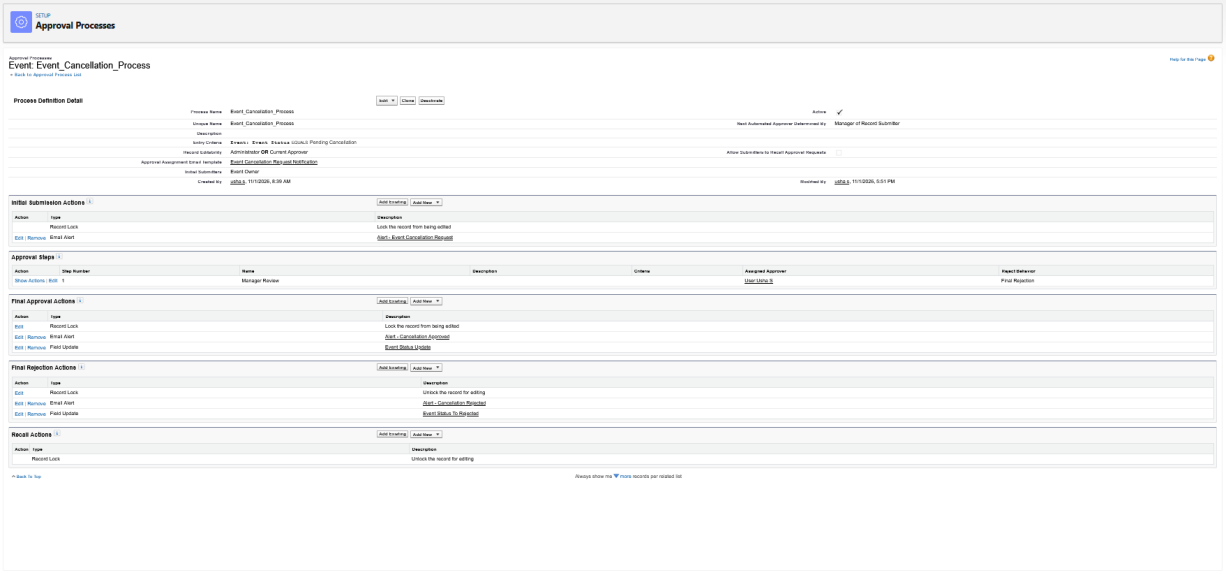


Figure 14: Active approval process with all actions

8. Flow: Client Reminder - 3 Days Before

A record-triggered flow runs on Event when the status is Confirmed. A scheduled path named 3_Days_Before waits until three days before the Event Date and then sends an email to the client with the event name, date, venue and type. The event owner receives a copy. The message subject and body are stored as text template resources.

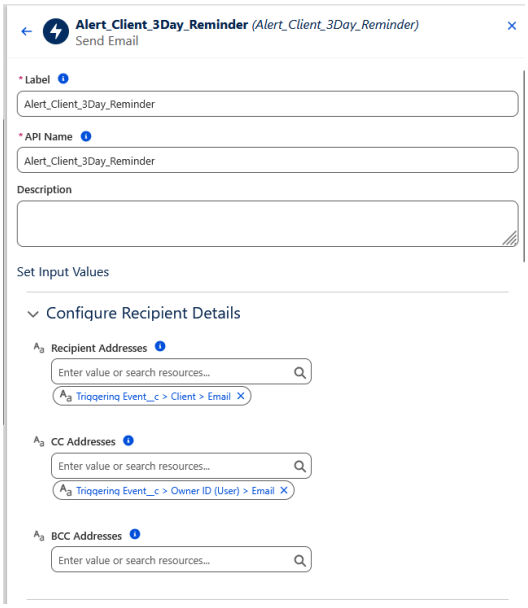


Figure 15: Send Email action (recipient is the client, copy to the owner)

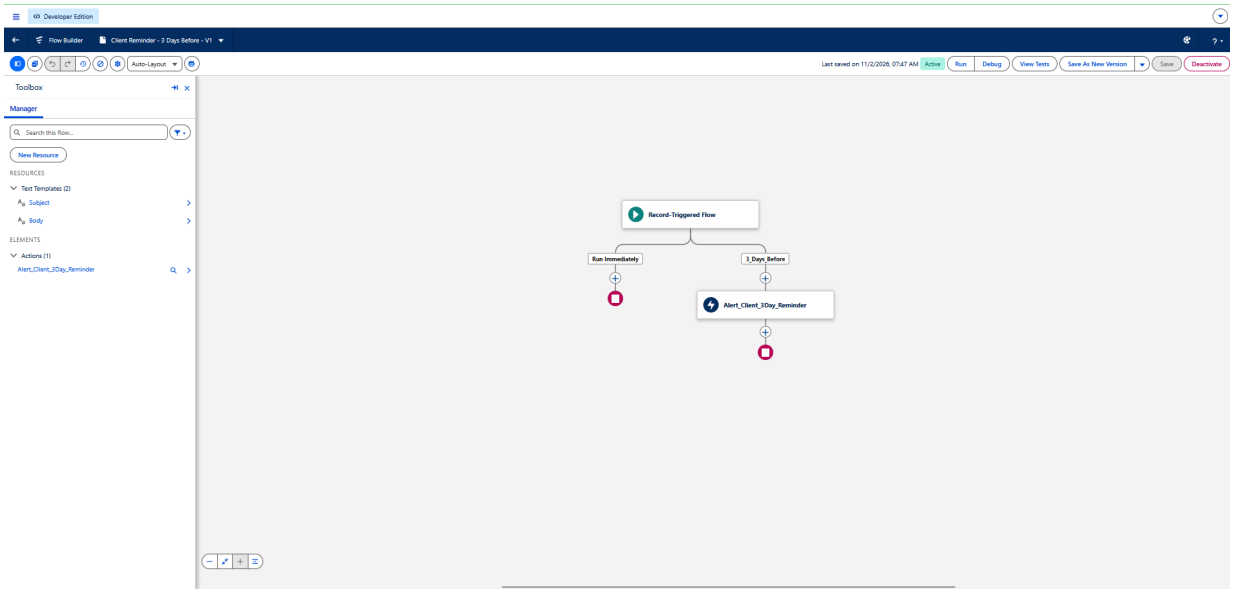


Figure 16: Flow in Flow Builder (active)

9. Apex code

9.1 PreventDoubleBooking (before insert and before update on Event)

The trigger collects the venues of the incoming events, loads the existing events for those venues in a single query, and adds an error when another event is booked at the same venue on the same date. This is bulk safe because the query is outside the loop.

```
trigger PreventDoubleBooking on Event__c (before insert, before update) {
    Set<Id> venueIds = new Set<Id>();
    for(Event__c ev : Trigger.new){
        if(ev.Venue__c != null) venueIds.add(ev.Venue__c);
    }
    Map<Id, List<Event__c>> venueEventMap = new Map<Id, List<Event__c>>();
    for(Event__c e : [SELECT Id, Venue__c, Event_Date__c FROM Event__c
                      WHERE Venue__c IN :venueIds]){
        if(!venueEventMap.containsKey(e.Venue__c)){
            venueEventMap.put(e.Venue__c, new List<Event__c>());
        }
        venueEventMap.get(e.Venue__c).add(e);
    }
    for(Event__c ev : Trigger.new){
        if(ev.Venue__c != null && venueEventMap.containsKey(ev.Venue__c)){
            for(Event__c existing : venueEventMap.get(ev.Venue__c)){
                if(existing.Event_Date__c == ev.Event_Date__c && existing.Id != ev.Id){
                    ev.addError('This Venue is already booked on this date.');
                }
            }
        }
    }
}
```

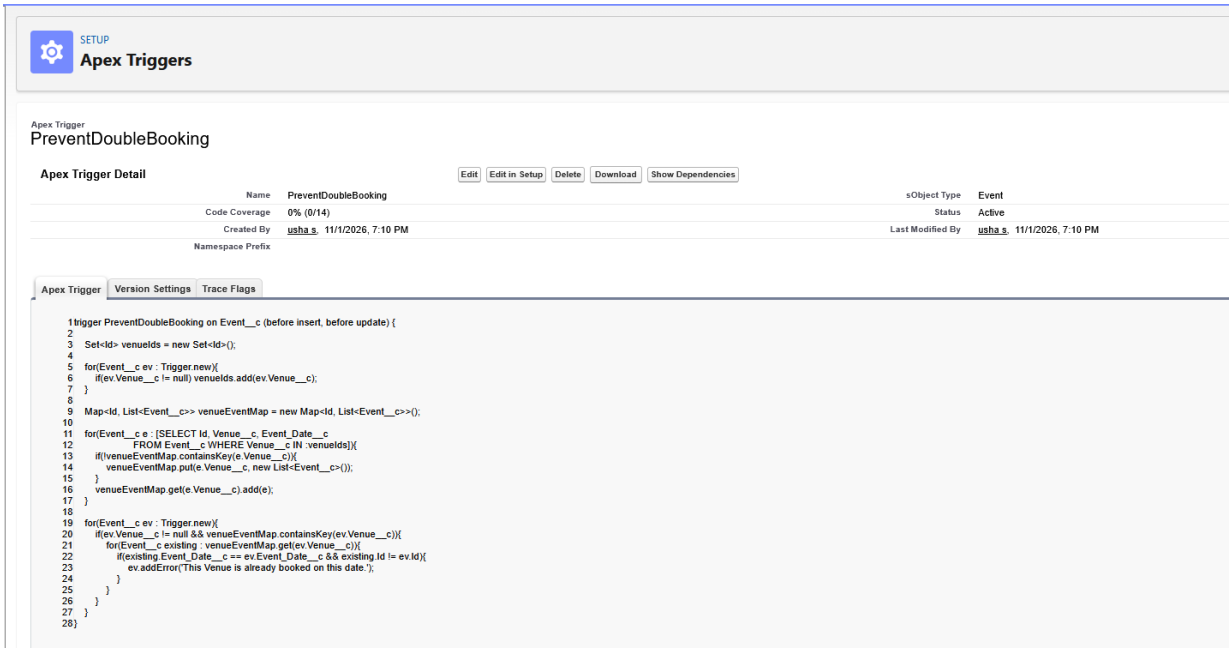


Figure 17: PreventDoubleBooking trigger in Salesforce

9.2 EventTrigger13 and VenueStatusHelper (after insert and after update on Event)

The trigger calls the helper class. A Confirmed event sets its venue to Reserved. A Canceled event sets its venue back to Available.

```
trigger EventTrigger13 on Event__c (after insert, after update) {
    VenueStatusHelper.updateVenueStatus(Trigger.new);
}

public with sharing class VenueStatusHelper {
    public static void updateVenueStatus(List<Event__c> events){
        List<Venue__c> venues = new List<Venue__c>();
        for(Event__c ev : events){
            if(ev.Venue__c != null){
                if(ev.Event_Status__c == 'Confirmed'){
                    venues.add(new Venue__c(Id=ev.Venue__c, Availability_Status__c='Reserved'));
                } else if(ev.Event_Status__c == 'Canceled'){
                    venues.add(new Venue__c(Id=ev.Venue__c, Availability_Status__c='Available'));
                }
            }
        }
        if(!venues.isEmpty()) update venues;
    }
}
```

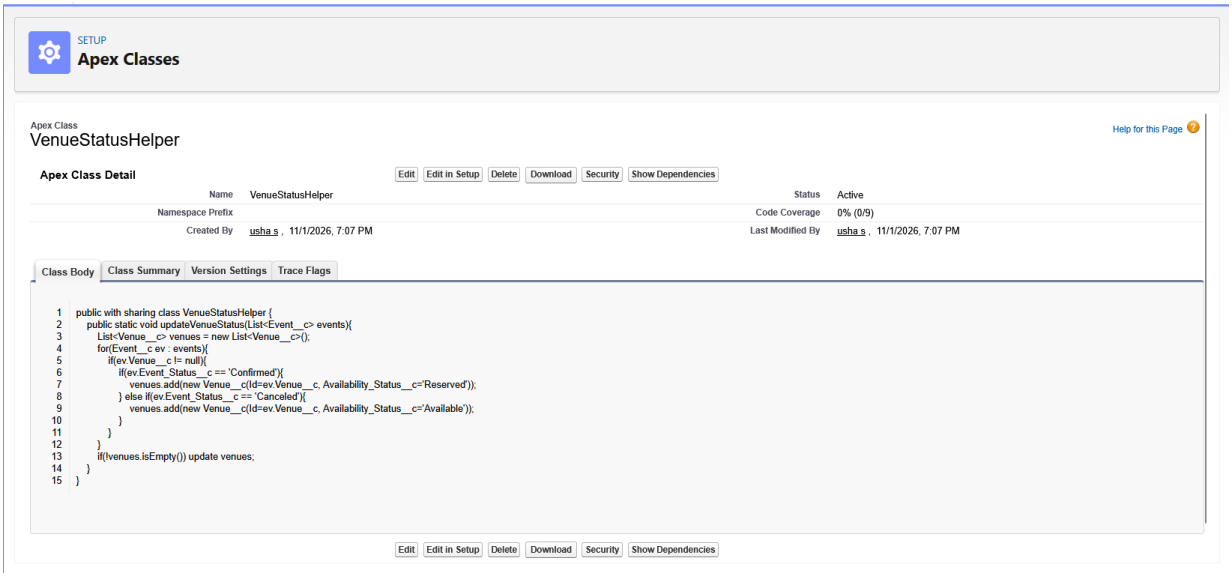


Figure 18: VenueStatusHelper class

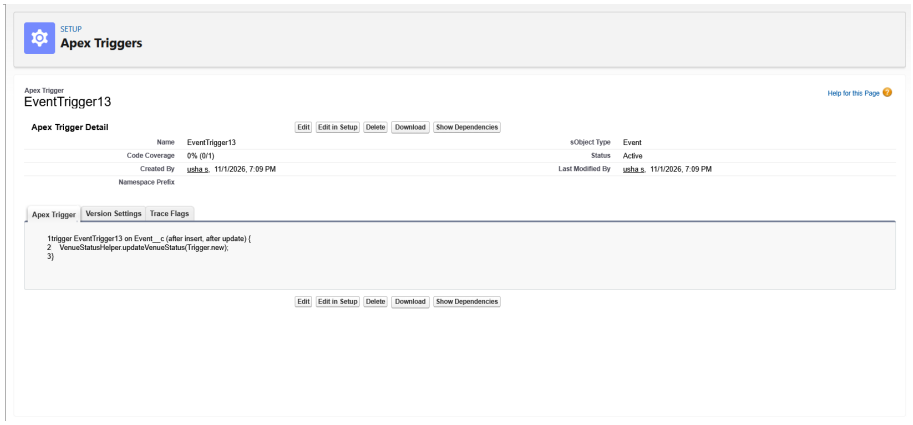


Figure 19: EventTrigger13 trigger

9.3 Batch Apex and Scheduled Apex

BatchCompleteEvents selects every event whose date is before today and whose status is not Completed, and sets the status to Completed. ScheduleCompleteEvents starts the batch. The job Daily Event Completion is scheduled to run every day at 8 PM.

```
global class BatchCompleteEvents implements Database.Batchable<sObject> {
    global Database.QueryLocator start(Database.BatchableContext bc){
        return Database.getQueryLocator('SELECT Id, Event_Date__c, Event_Status__c FROM Event__c ' +
            'WHERE Event_Date__c < TODAY AND Event_Status__c != \'Completed\'');
    }
    global void execute(Database.BatchableContext bc, List<Event__c> scope){
        for(Event__c ev : scope){ ev.Event_Status__c = 'Completed'; }
        update scope;
    }
    global void finish(Database.BatchableContext bc){ System.debug('Past events marked as Completed'); }
}

global class ScheduleCompleteEvents implements Schedulable {
    global void execute(SchedulableContext sc){ Database.executeBatch(new BatchCompleteEvents(), 200); }
}
```

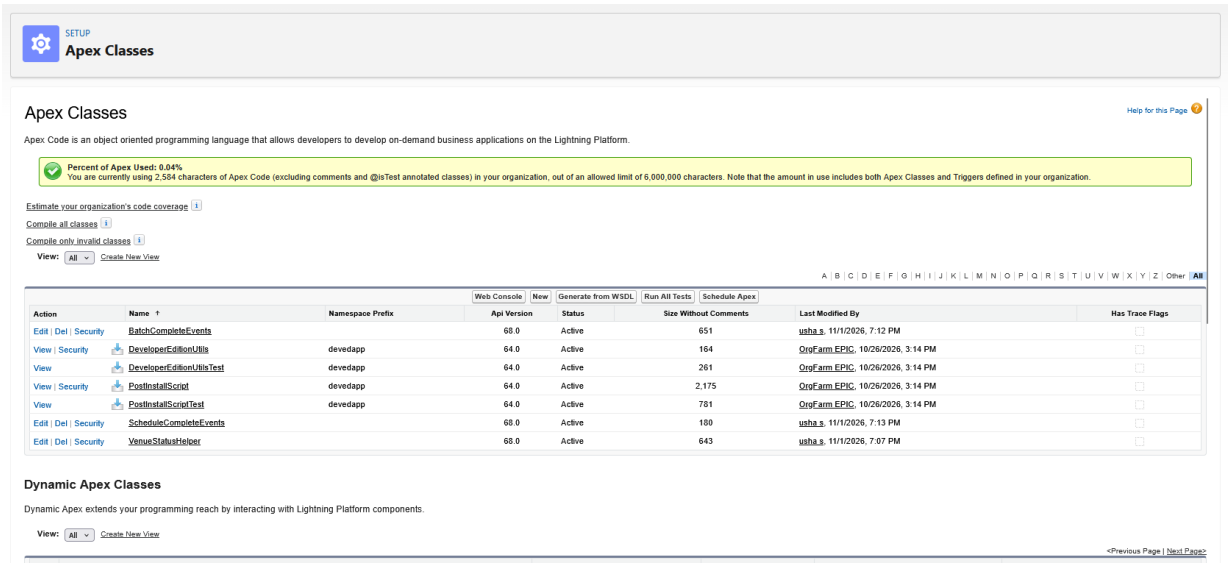


Figure 20: Apex classes (batch, schedulable and helper)

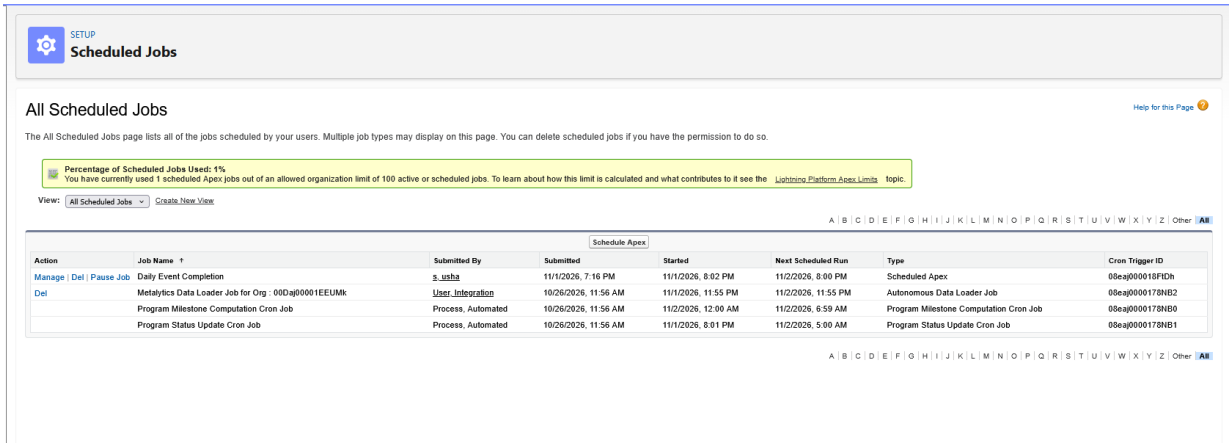


Figure 21: Scheduled job Daily Event Completion

Development Phase 3: UI/UX Development and Customization

This phase gave users a simple place to work and gave management a clear view of the data.

1. Lightning app: Event Planner

A Lightning app named Event Planner was created in App Manager. Its navigation bar contains Events, Clients, Vendors, Venues, Feedbacks, Reports, Dashboards and Event Vendors. The app is made visible to the System Administrator and to the four custom profiles, and it is set as the default app of the custom profiles so that users land in it after login.

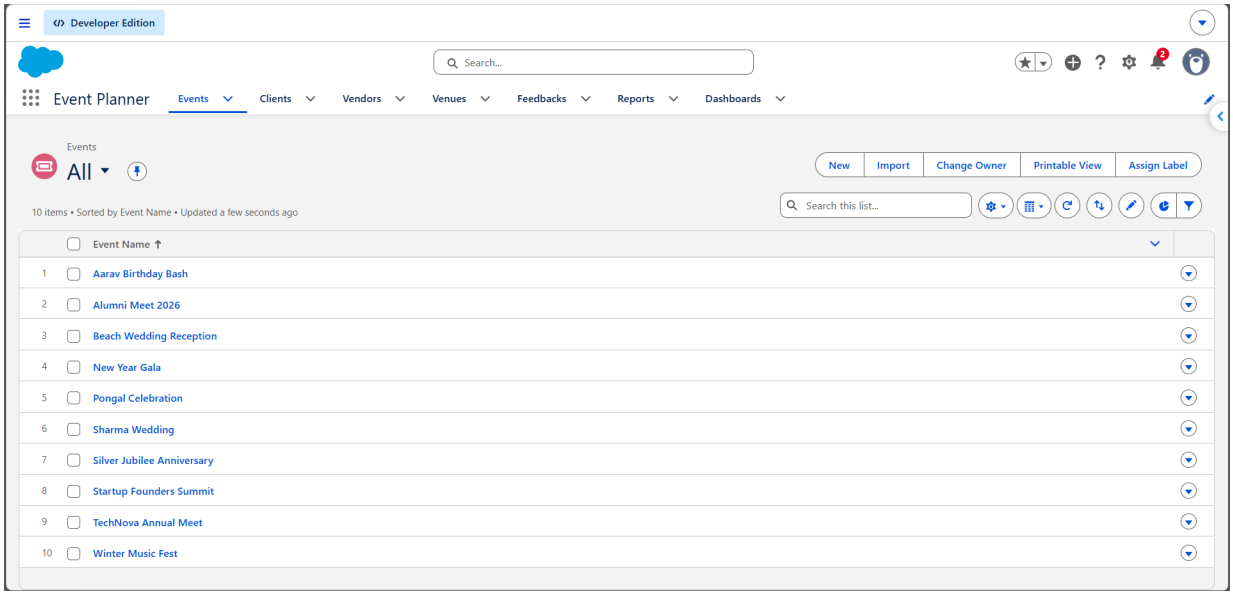


Figure 1: Event Planner app with the Events list

2. Report: Upcoming Events by Month

Setting	Value
Report type	Events
Columns	Event Name, Event Date, Event Type, Client, Venue, Event Budget
Filters	Show Me: All events; Event Date greater than or equal to today; Event Status not equal to Canceled
Grouping	Event Date by calendar month (row counts and budget subtotal per month)
Chart	Sum of Event Budget by month
Folder	EventForce Report, shared as viewer with the Event Coordinator and Vendor Manager roles

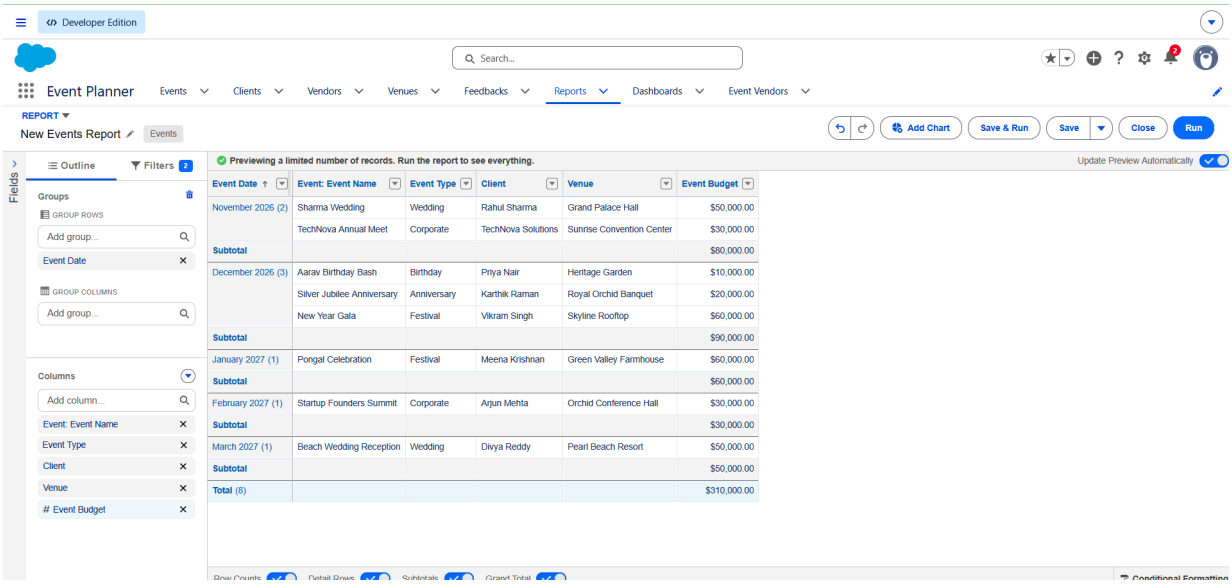


Figure 2: Report grouped by month with subtotals

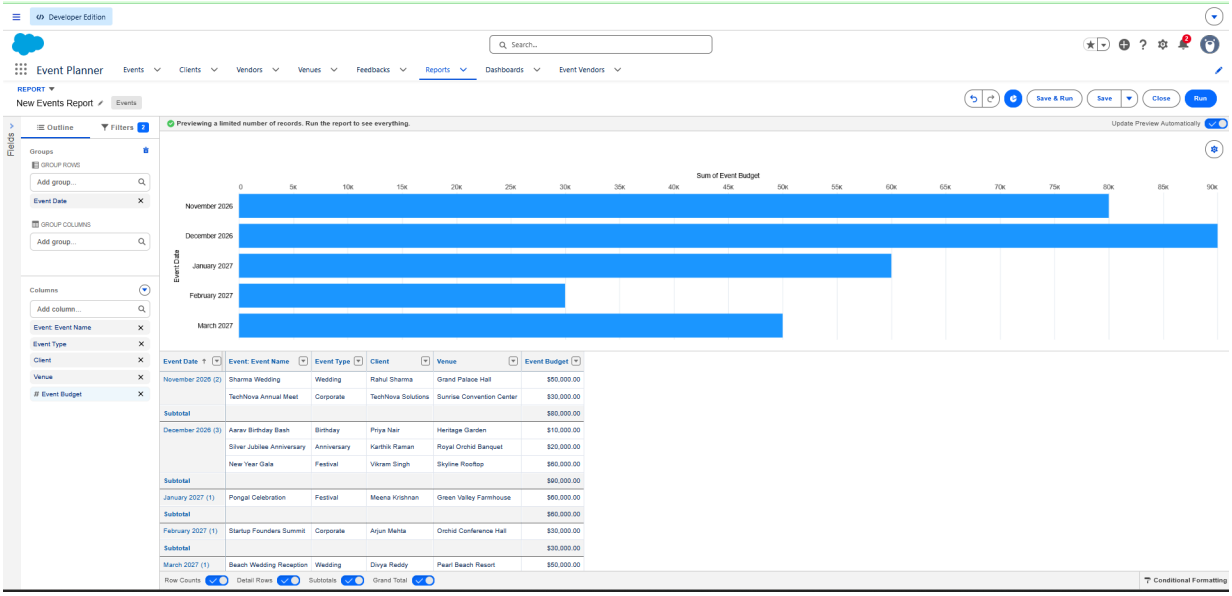


Figure 3: Report with chart

3. Dashboard: EventForce Operations Dashboard

The dashboard shows the report as a donut chart titled Upcoming Events Overview. Each slice is the planned budget of one month, and the centre shows the total planned budget of upcoming events. It is built from the report so the numbers always match the report.

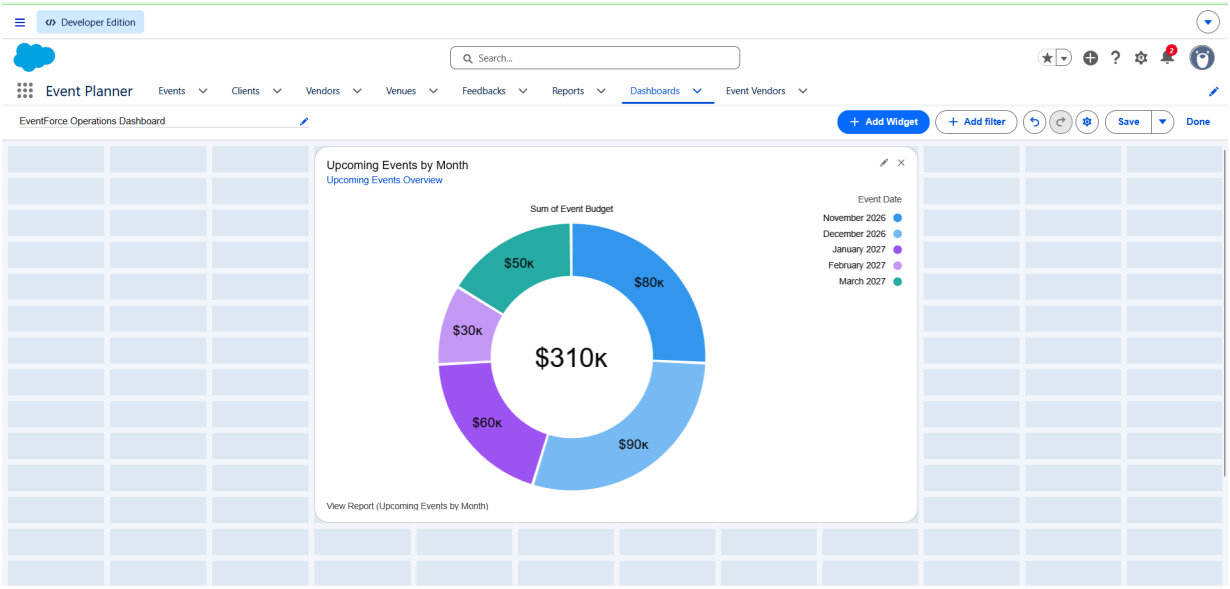


Figure 4: Operations dashboard

4. Usability decisions

- One app contains every object, report and dashboard, so users do not have to search for tabs.
- Picklists are used for type, status, country and city to avoid typing mistakes.
- The Event record shows the formula budget automatically so coordinators do not calculate it by hand.

Development Phase 4: Data Migration, Testing and Security

1. Data migration

Sample data was prepared as five CSV files (ten records each) and loaded with the Data Import Wizard in the order Venues, Clients, Vendors, Events. This order matters because events refer to venues and clients.

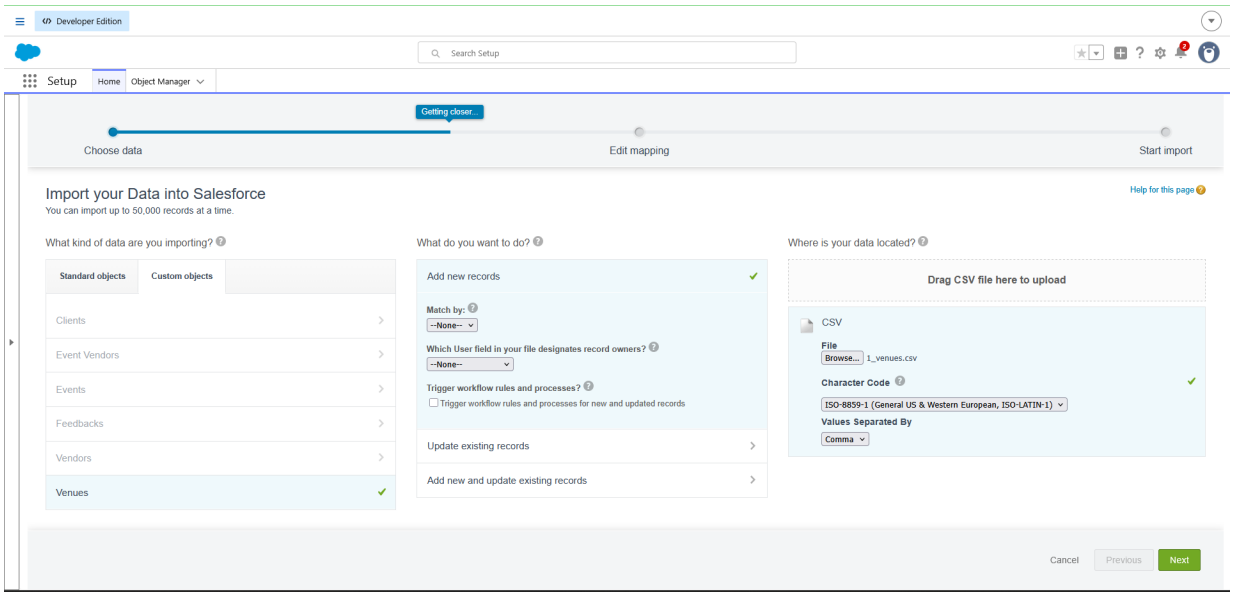


Figure 1: Data Import Wizard (custom object Venues, add new records)

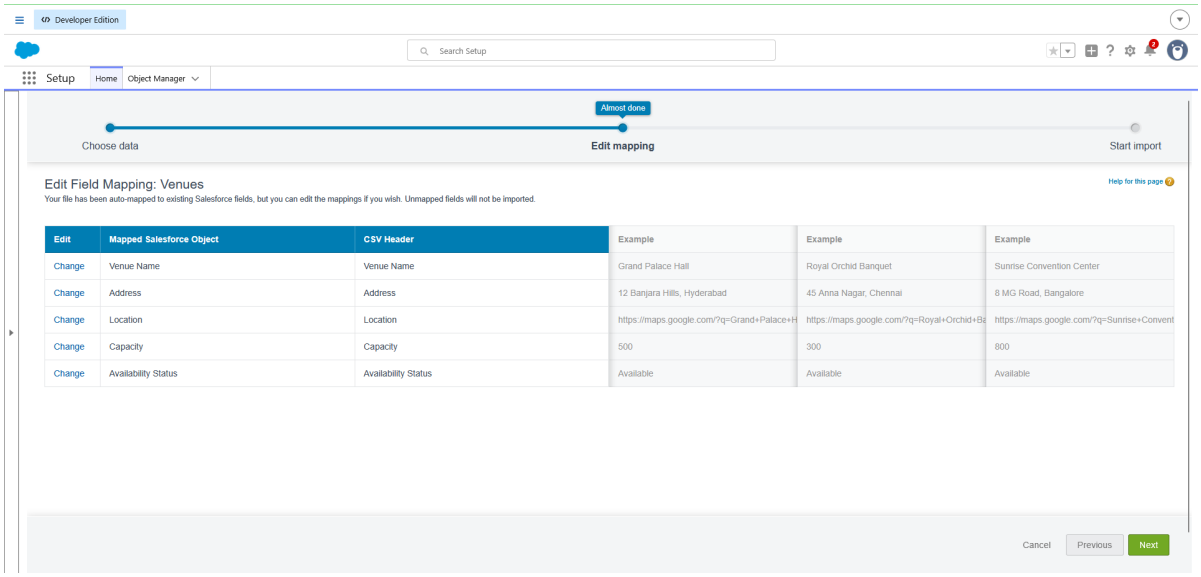


Figure 2: Field mapping for Venues

The wizard cannot map custom lookup fields, so after importing the events, the Venue and Client lookups were filled by an anonymous Apex script that matches records by name. Feedback records were created by a second script that links each feedback to its event and client. The scripts are in the apex-code folder of the repository.

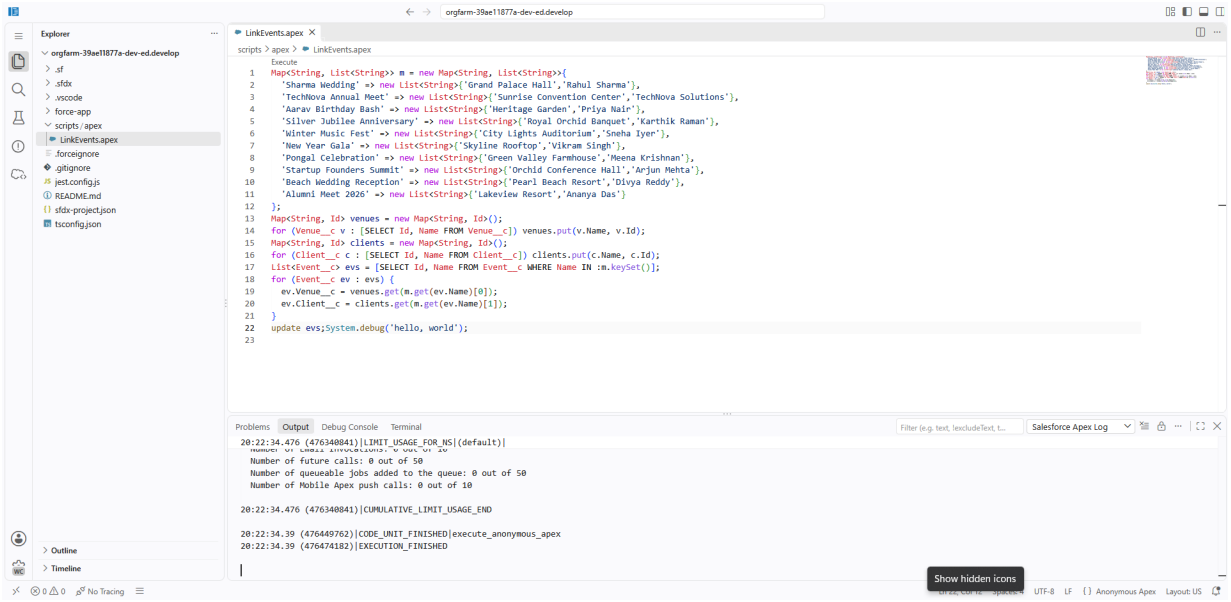


Figure 3: Script that links events to venues and clients (execution finished)

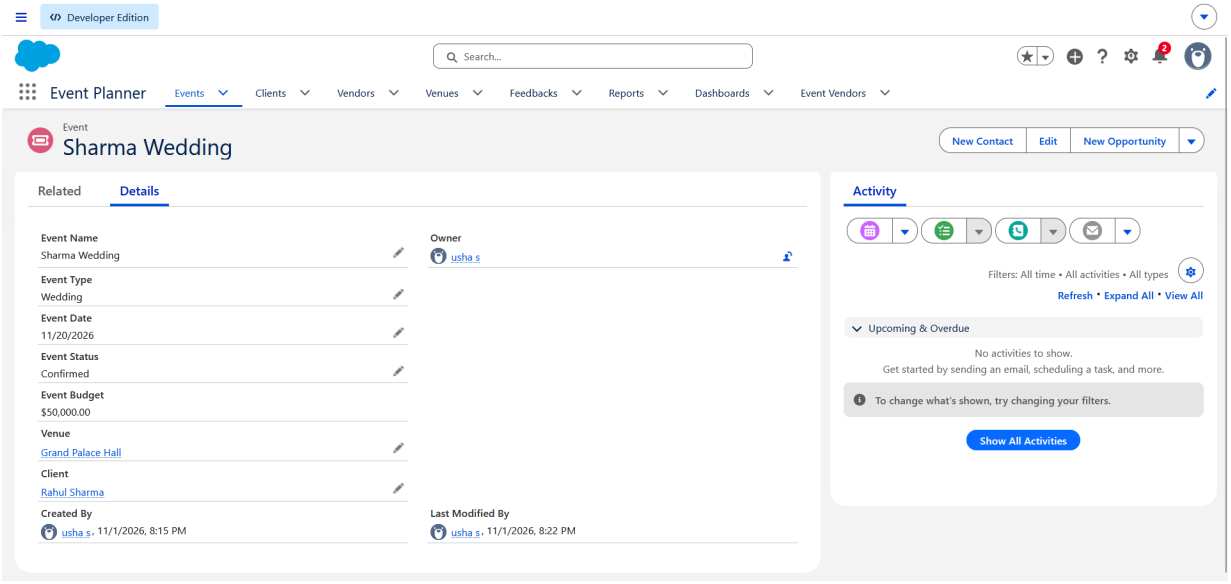


Figure 4: Event record after linking (venue, client and formula budget)

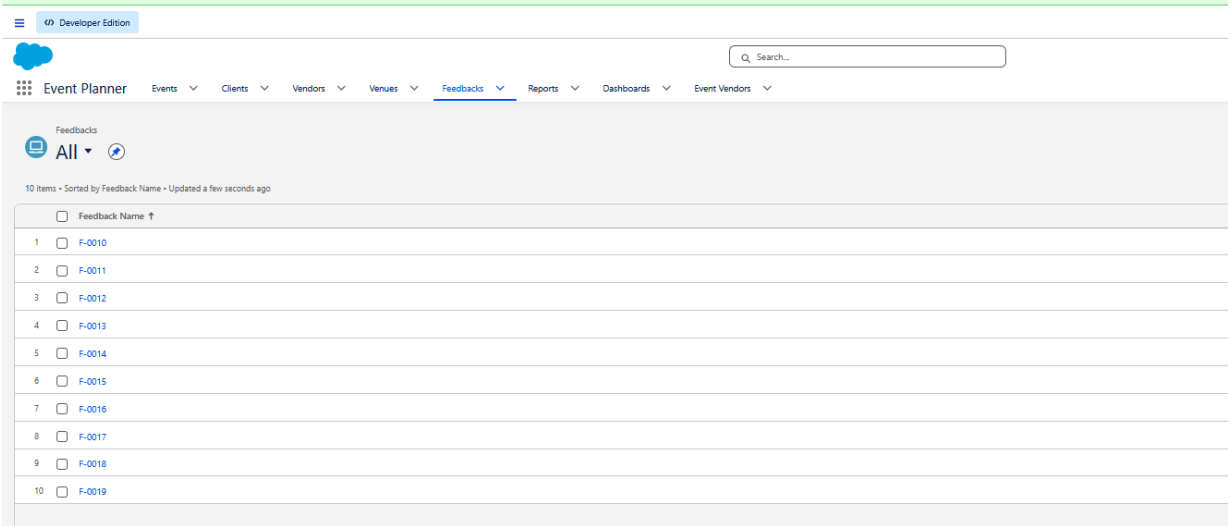


Figure 5: Ten feedback records

Data verification

Object	Records	Check
Venue	10	Names, capacity and status loaded correctly
Client	10	All emails passed the validation rule
Vendor	10	Multi-select service types loaded
Event	10	Venue, client and formula budget filled; Confirmed events set their venues to Reserved
Feedback	10	Each feedback linked to an event and the matching client

2. Roles and users

The role hierarchy has Event Admin on top and Event Coordinator, Vendor Manager and Client below it. Four users were created, one for each role. Non admin users use the Salesforce Platform license because the Developer org has only a small number of full licenses.

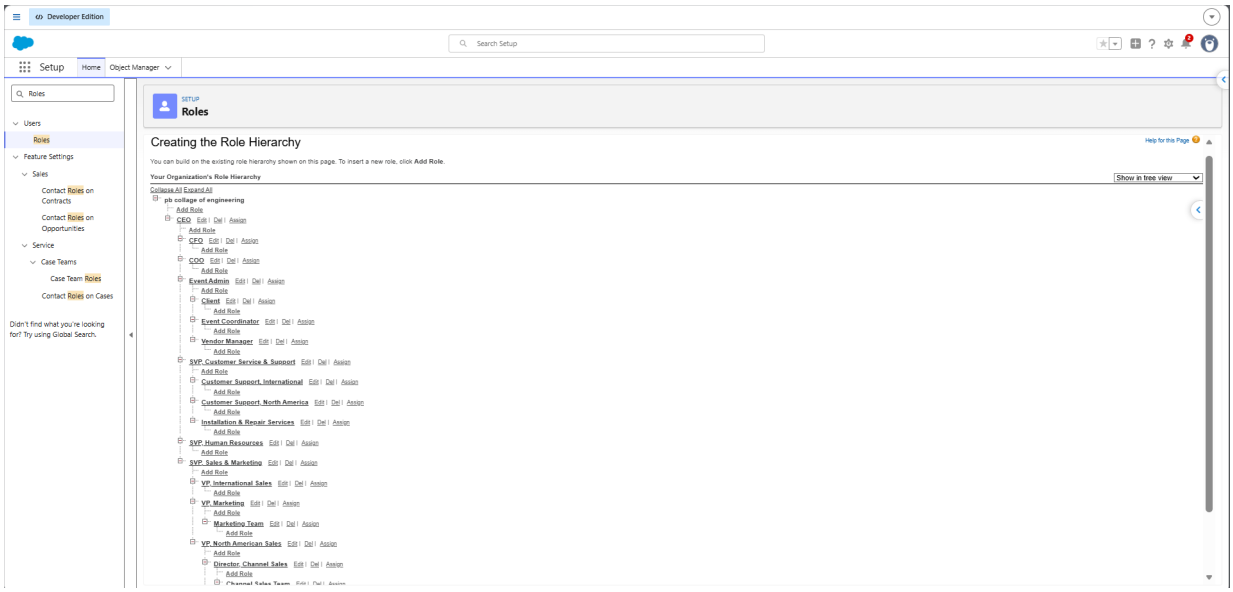


Figure 6: Role hierarchy

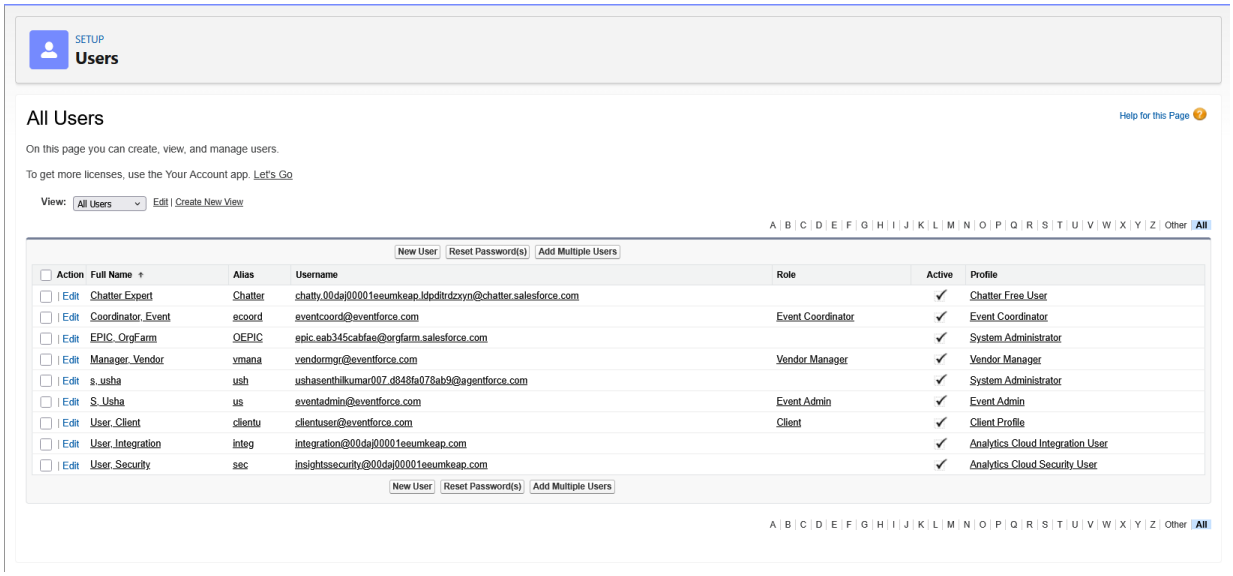


Figure 7: Users with their roles and profiles

3. Profiles

Profile	Based on	Object access
Event Admin	System Administrator	Full access
Event Coordinator	Standard Platform User	Event full; Client read and edit; Vendor and Venue read; Feedback create and read
Vendor Manager	Standard Platform User	Vendor full; Event, Venue, Client and Feedback read
Client Profile	Standard Platform User	Event read; Feedback create and read; no Vendor or Venue access

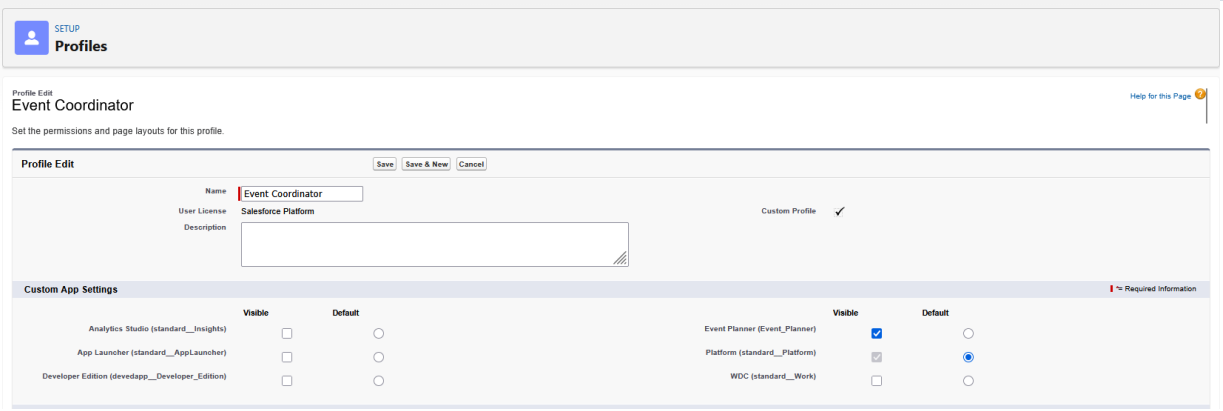


Figure 8: Event Planner set as the default app on a profile

4. Permission set

The permission set Feedback Manager gives Read, Create, Edit and Delete on Feedback. It is assigned to the Event Coordinator user, so only the coordinator can manage feedback records fully.

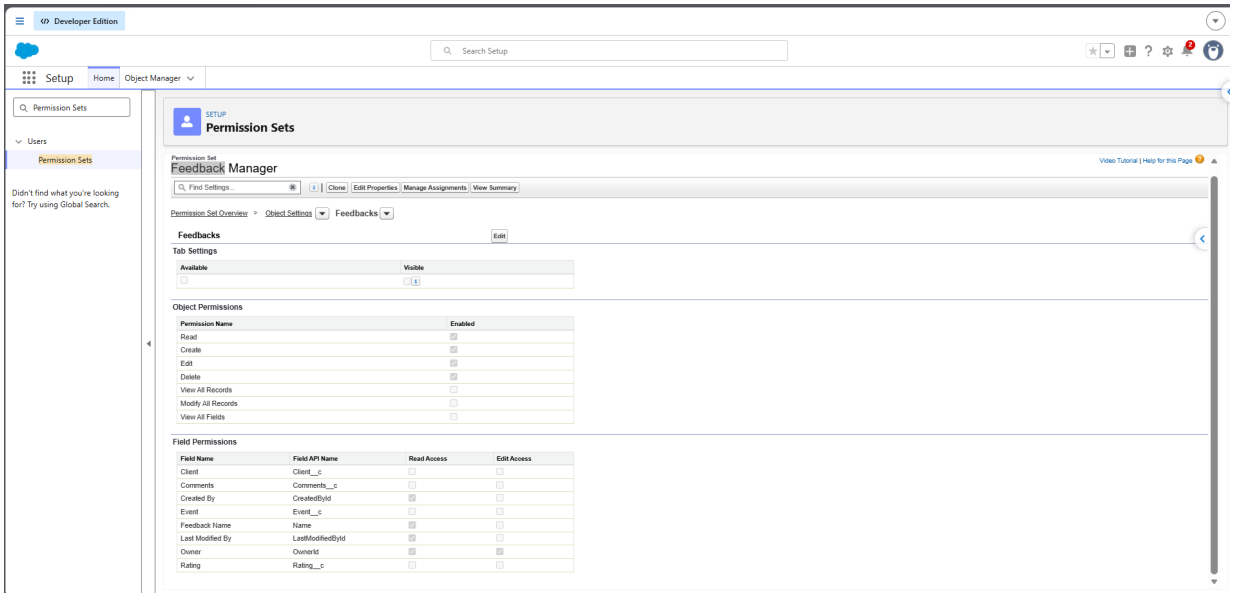


Figure 9: Feedback Manager permission set

5. Organization wide defaults and sharing rule

The default internal and external access of Event was changed to Private, so users see only their own events and those of people below them in the hierarchy. The sharing rule Event_Sharing_For_Vendors shares events owned by the Event Coordinator role with the Vendor Manager role as read only, so vendor managers can see which events need vendors without being able to change them.

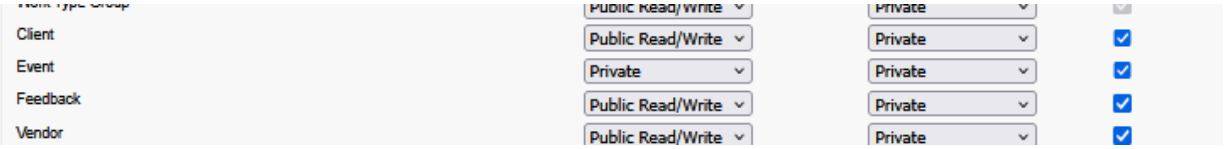


Figure 10: Event is Private

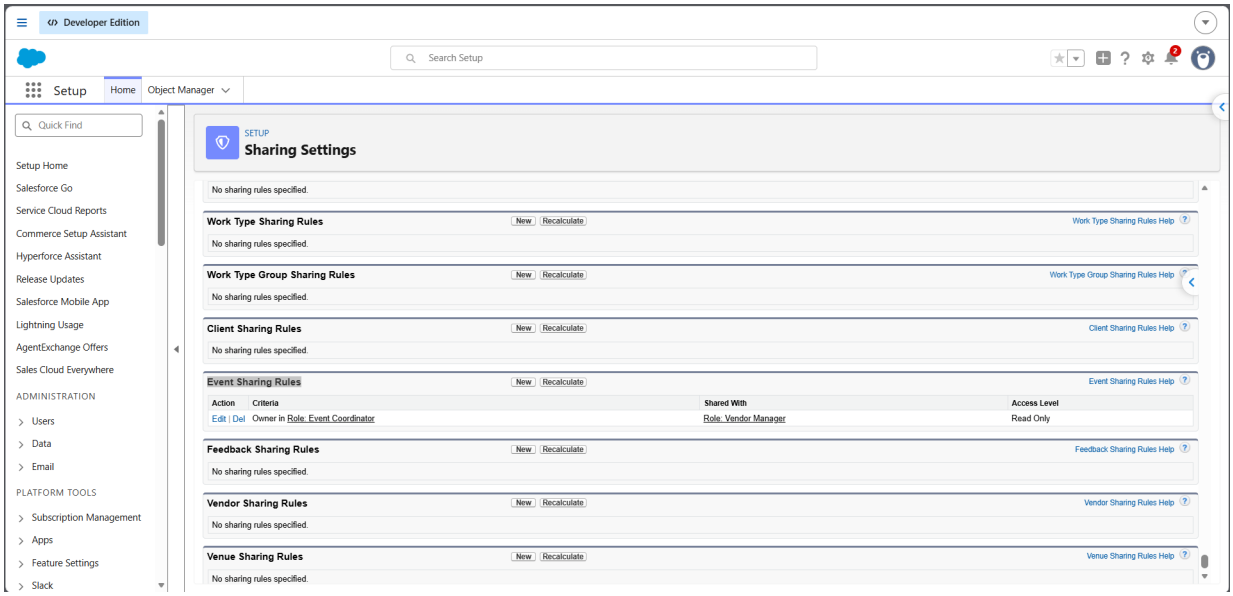


Figure 11: Sharing rule for Event

6. Testing of data and security

- Imported data was opened in the app to confirm the counts and the links between objects.
- Validation rule, trigger and approval behaviour were tested after the import (see the testing section of the report).
- Access was reviewed profile by profile against the access matrix.

Testing Results and Conclusion

1. Test cases and results

ID	Feature	Steps	Expected	Result
TC01	Validation rule	Create a Client with email test-client@gmail.com	Error: Please Enter Valid Email Address	Pass
TC02	PreventDoubleBooking trigger	Create an Event at Grand Palace Hall on 11/20/2026 (already booked)	Error: This Venue is already booked on this date.	Pass
TC03	Venue status trigger	Open Grand Palace Hall (event Confirmed)	Availability Status is Reserved	Pass
TC04	Approval process	Set Pongal Celebration to Pending Cancellation, submit, approve	Approval History Approved and Event Status Canceled	Pass
TC05	Scheduled Apex	Open Scheduled Jobs	Daily Event Completion is scheduled for 8 PM	Pass

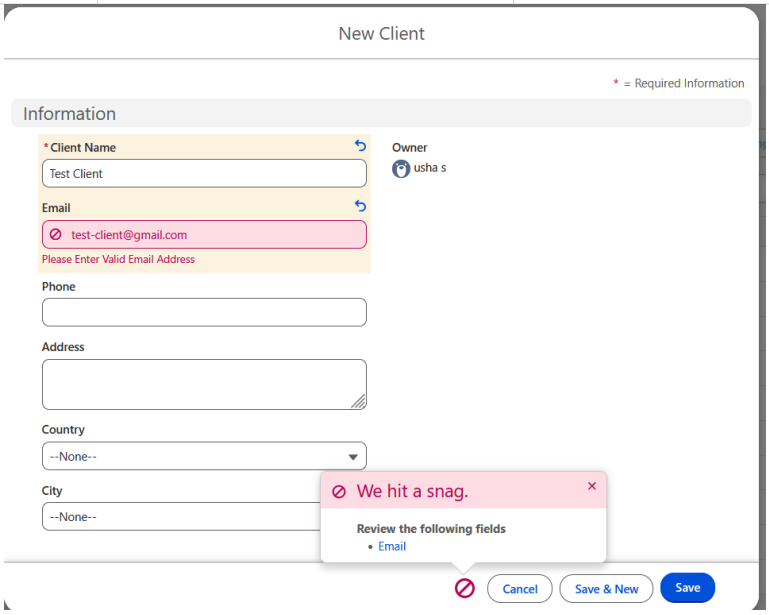


Figure 1 (TC01): Validation rule error

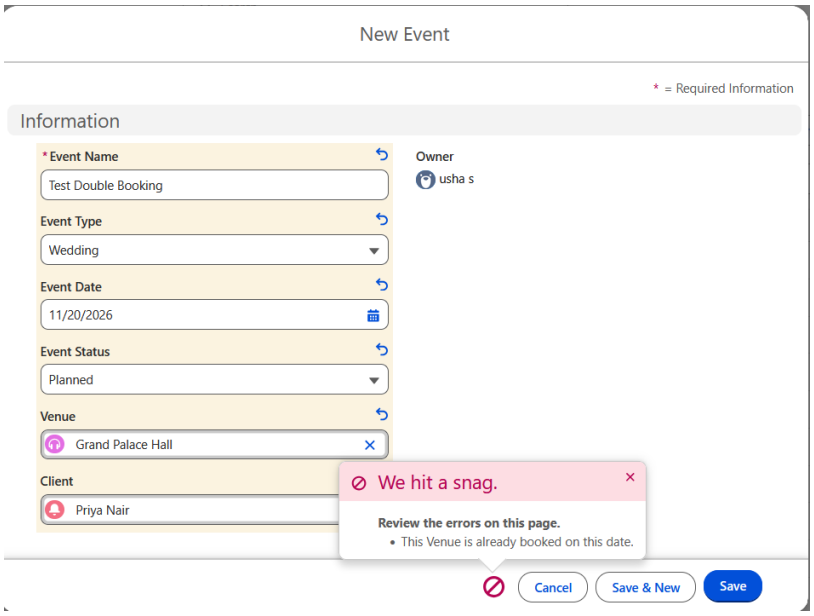


Figure 2 (TC02): Double booking blocked

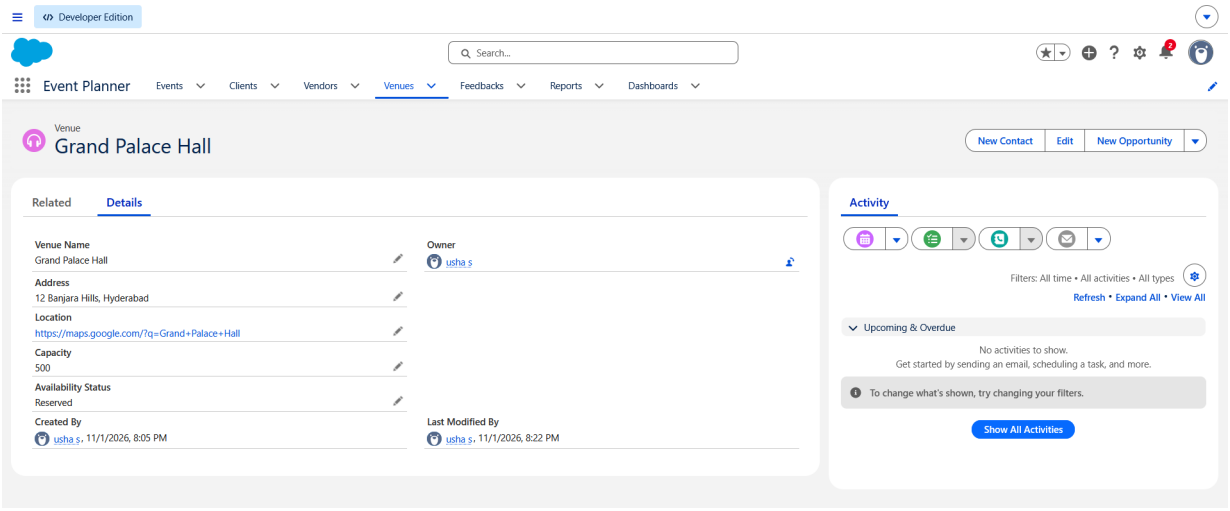


Figure 3 (TC03): Venue Reserved

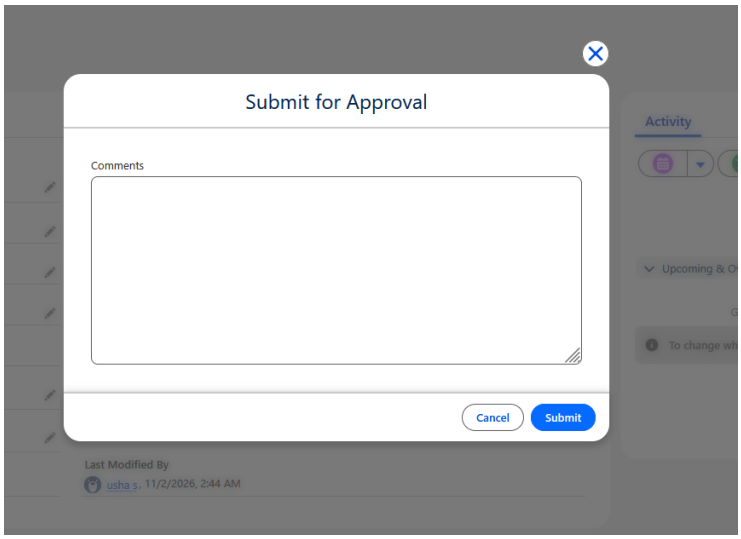


Figure 4 (TC04): Submit for Approval

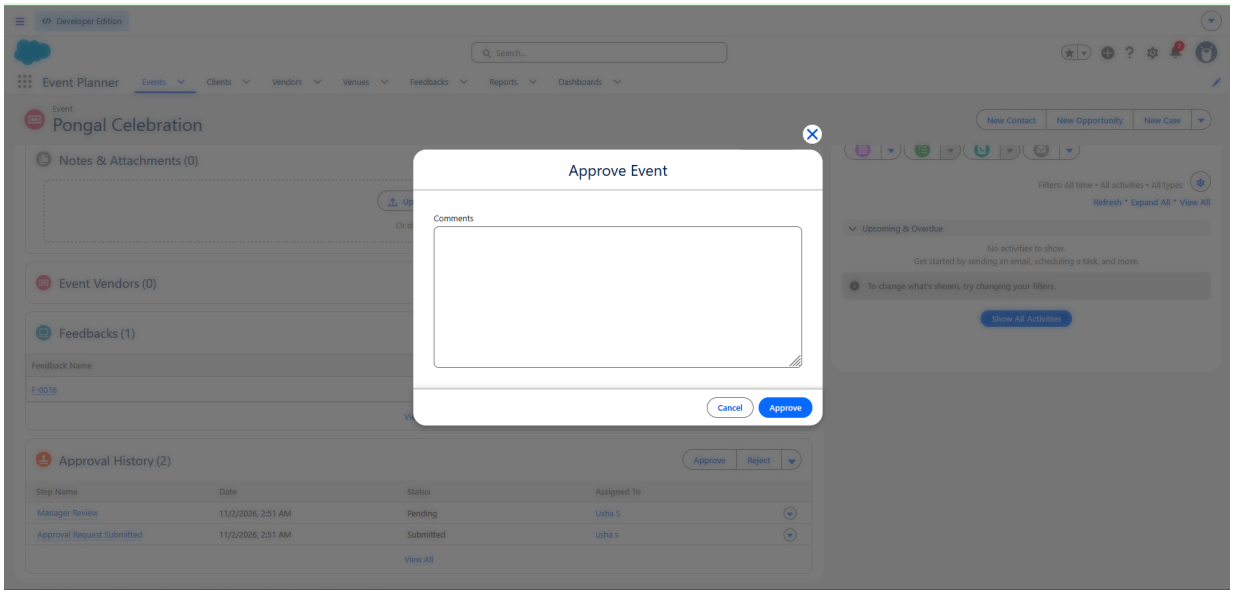


Figure 5 (TC04): Approve Event

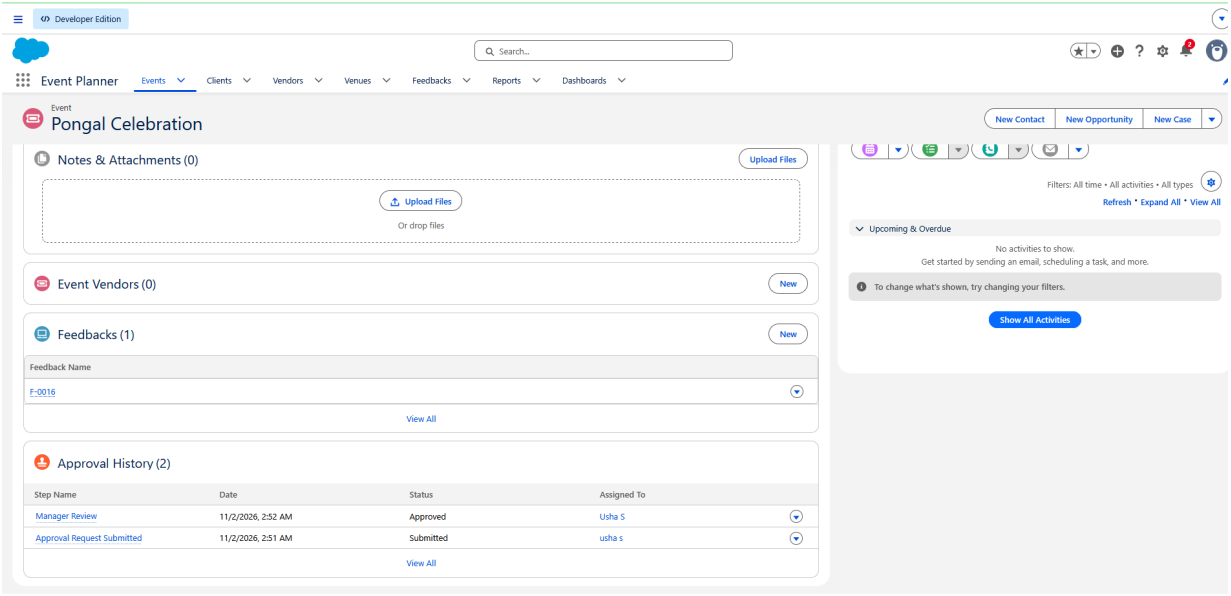


Figure 6 (TC04): Approval history shows Approved

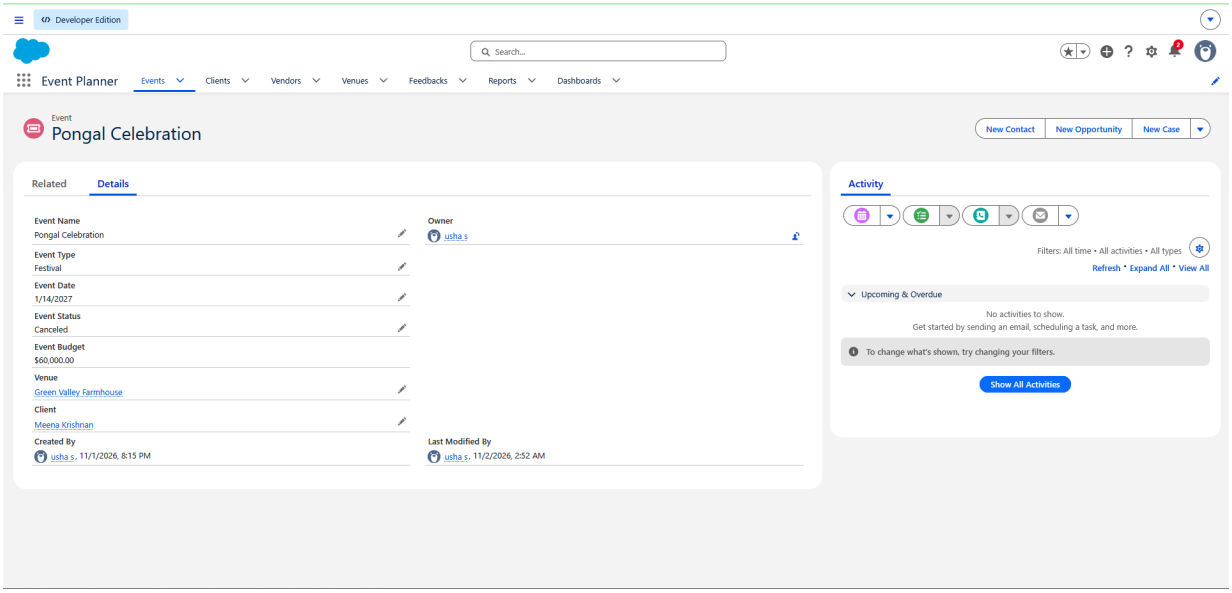


Figure 7 (TC04): Event Status is Canceled

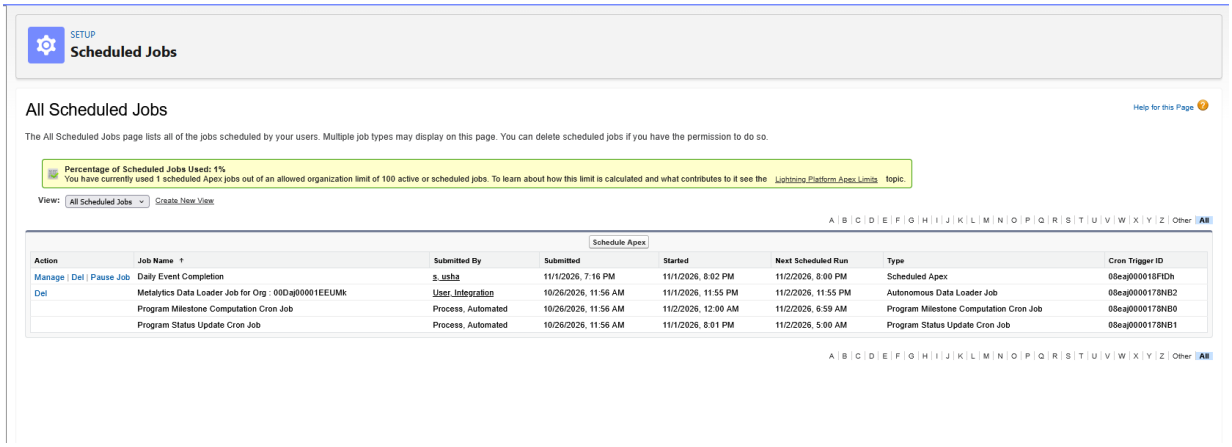


Figure 8 (TC05): Scheduled job listed

Other checks: the Event Budget formula returns 50,000 for a Wedding and 60,000 for a Festival; the report and dashboard show upcoming events grouped by month and exclude canceled and past events.

2. Advantages

- One system for events, clients, venues, vendors and feedback.
- Double booking is prevented by the system, not by memory.
- Venue availability stays correct automatically.
- Clients are reminded without manual work.
- Cancellations are approved, recorded and communicated.
- Access follows the principle of least privilege.
- Management gets a live monthly view of events and budget.

3. Disadvantages and limitations

- Built and tested in a Developer Edition org with limited licenses and sample data only.
- Reminder emails go to sample addresses, so real delivery was not verified.
- Vendors are linked to events through the junction object, but no automatic vendor availability check is included.
- No integration with payment, calendar or messaging services.

4. Conclusion

EventForce shows how the Salesforce platform can turn a spreadsheet based event business into a controlled, automated and measurable process. The team built a complete data model, enforced business rules with validation and Apex, automated reminders and cancellations, secured the data with roles, profiles, a permission set and sharing, and delivered an app, a report and a dashboard. All planned test cases passed. The project also gave every member practical experience with objects, flows, approvals, Apex, security and data loading.

5. Future scope

- Deploy the solution to a production org with change sets or Salesforce CLI.
- Add vendor availability checks and vendor performance tracking.
- Add more reports and dashboards (venue usage, vendor workload, feedback ratings).
- Integrate with calendar, messaging and payment services.
- Provide a mobile friendly experience for coordinators on site.
- Add automated Apex test classes for deployment readiness.

6. Appendix

Area	Items
Custom objects	Event, Client, Vendor, Venue, Event Vendor, Feedback
Automation	Validation rule, approval process, flow, Apex triggers and classes
Processing	Batch Apex and Scheduled Apex
Security	Profiles, roles, permission set, organization wide defaults, sharing rule
Data	Data Import Wizard files and Apex scripts in the repository

Area	Items
UI and analytics	Event Planner app, report and dashboard

Team

Name	Role
Monika P	Team Leader
Usha S	Team Member
Shobana M	Team Member
Mujahidha Begam M	Team Member